

Documentation Indicators

Indicators

Class **IndicatorFactory** is a one of the most powerful entities in the vectorbt's ecosystem - it can wrap any indicator function and make it parametrizable and analyzable.

Pipeline

An indicator is a pipeline that does the following:

- Accepts input arrays (for example, the opening and closing price)
- Accepts parameters either in a scalar or array format (for example, window size)
- Accepts other relevant arguments and keyword arguments
- Broadcasts input arrays against each other or some shape
- Broadcasts parameters against each other to form a fixed set of parameter combinations
- For each parameter combination, performs calculation on the input arrays to produce output arrays of the same shape (for example, rolling average)
- Concatenates output arrays of all parameter combinations along columns
- Converts the results back into the Pandas format

Let's manually create an indicator that takes two time series, calculates their normalized moving averages, and returns the difference of both. We'll test different shapes as well as parameter combinations to take advantage of broadcasting:

```
>>> from vectorbtpro import *

>>> def mov_avg_crossover(ts1, ts2, w1, w2):
...     ts1, ts2 = vbt.broadcast(ts1, ts2) ❶
...
...     w1, w2 = vbt.broadcast( ❷
...         vbt.to_1d_array(w1),
...         vbt.to_1d_array(w2))
...
...     ts1_mas = []
...     for w in w1:
...         ts1_mas.append(ts1.vbt.rolling_mean(w) / ts1) ❸
...     ts2_mas = []
...     for w in w2:
...         ts2_mas.append(ts2.vbt.rolling_mean(w) / ts2)
...
...     ts1_ma = pd.concat(ts1_mas, axis=1) ❹
...     ts2_ma = pd.concat(ts2_mas, axis=1)
...
...     ts1_ma.columns = vbt.combine_indexes(( ❺
...         pd.Index(w1, name="ts1_window"),
...         ts1.columns))
...     ts2_ma.columns = vbt.combine_indexes((
...         pd.Index(w2, name="ts2_window"),
...         ts2.columns))
...
...     return ts1_ma.vbt - ts2_ma ❻

>>> def generate_index(n): ❼
...     return vbt.date_range("2020-01-01", periods=n)

>>> ts1 = pd.Series([1, 2, 3, 4, 5, 6, 7], index=generate_index(7))
>>> ts2 = pd.DataFrame({
...     'a': [5, 4, 3, 2, 3, 4, 5],
...     'b': [2, 3, 4, 5, 4, 3, 2]
... }, index=generate_index(7))
>>> w1 = 2
>>> w2 = [3, 4]

>>> mov_avg_crossover(ts1, ts2, w1, w2)
ts1_window      2
ts2_window      3      4
              a      b      a      b
2020-01-01      NaN      NaN      NaN      NaN
2020-01-02      NaN      NaN      NaN      NaN
2020-01-03 -0.500000  0.083333      NaN      NaN
2020-01-04 -0.625000  0.075000 -0.875000  0.175000
2020-01-05  0.011111 -0.183333 -0.100000 -0.100000
2020-01-06  0.166667 -0.416667  0.166667 -0.416667
2020-01-07  0.128571 -0.571429  0.228571 -0.821429
```

- ❶ Both time series become Pandas objects with the same shape - (7, 2)
- ❷ Window `w1` becomes [2, 2] and window `w2` becomes [3, 4]. This builds two parameter combinations: (2, 3) and (2, 4).
- ❸ Calculate the normalized moving average based on each of the windows
- ❹ Concatenate the DataFrames along the column axis
- ❺ Create a new column hierarchy with the window values on top

- ⑥ Calculate the difference of both concatenated DataFrames
- ⑦ Convenient function to generate a datetime-like index

Neat! We just created a pretty flexible pipeline that takes arbitrary input and parameter combinations. The end result of this pipeline is a DataFrame where each column corresponds to a single window combination applied on a single column in both `ts1` and `ts2`. But is this pipeline user-friendly? 😞 Having to deal with broadcasting, output concatenation, and the column hierarchy makes this code no different from a regular Pandas code.

The pipeline above can be well standardized, which is done by `IndicatorBase.run_pipeline`. It conveniently prepares inputs, parameters, and columns, while the calculation and output concatenation have to be performed by the user using `custom_func`. Let's rewrite the above example a bit:

```
>>> def custom_func(ts1, ts2, w1, w2):
...     ts1_mas = []
...     for w in w1:
...         ts1_mas.append(vbt.nb.rolling_mean_nb(ts1, w) / ts1) ①
...     ts2_mas = []
...     for w in w2:
...         ts2_mas.append(vbt.nb.rolling_mean_nb(ts2, w) / ts2)
...
...     ts1_ma = np.column_stack(ts1_mas) ②
...     ts2_ma = np.column_stack(ts2_mas)
...
...     return ts1_ma - ts2_ma ③

>>> outputs = vbt.IndicatorBase.run_pipeline(
...     num_ret_outputs=1,
...     custom_func=custom_func,
...     inputs=dict(ts1=ts1, ts2=ts2),
...     params=dict(w1=w1, w2=w2)
... )
>>> outputs
(<vectorbtpro.base.wrapping.ArrayWrapper at 0x7fb188993160>,
 [array([[1, 1],
        [2, 2],
        [3, 3],
        [4, 4],
        [5, 5],
        [6, 6],
        [7, 7]]),
 array([[5, 2],
        [4, 3],
        [3, 4],
        [2, 5],
        [3, 4],
        [4, 3],
        [5, 2]]),
 array([0, 1, 0, 1]),
 [],
 [array([[ nan,      nan,      nan,      nan],
        [ nan,      nan,      nan,      nan],
        [-0.5,      0.08333333,      nan,      nan],
        [-0.625,      0.075,      -0.875,      0.175],
        [ 0.01111111,      -0.18333333,      -0.1,      -0.1],
        [ 0.16666667,      -0.41666667,      0.16666667,      -0.41666667],
        [ 0.12857143,      -0.57142857,      0.22857143,      -0.82142857]]),
 [[2, 2], [3, 4]],
 [Int64Index([2, 2, 2, 2], dtype='int64'),
  Int64Index([3, 3, 4, 4], dtype='int64')],
 [])
```

- ① Using `rolling_mean_nb`
- ② Concatenate the NumPy arrays along the column axis
- ③ Calculate the difference of both concatenated NumPy arrays

We produced much less code and did the entire calculation using NumPy and Numba alone - a big win! But what is this monstrous output?

This raw output is really meant to be used by vectorbt, not by the user themselves - it contains useful metadata for working the indicator. Additionally, if you look into the source of this function, you will notice that it accepts a ton of arguments. Great complexity enables great flexibility: each argument is targeted at configuring a specific step of the pipeline. But don't worry: we won't use this function directly.

Factory

Instead, we will use `IndicatorFactory`, which simplifies the usage of `IndicatorBase.run_pipeline` by providing a unified interface and various automations. Let's wrap our `custom_func` using the factory:

```
>>> MADiff = vbt.IF(
...     class_name='MADiff',
...     input_names=['ts1', 'ts2'],
...     param_names=['w1', 'w2'],
...     output_names=['diff'],
... ).with_custom_func(custom_func)

>>> madiff = MADiff.run(ts1, ts2, w1, w2)
>>> madiff.diff
madiff_w1
madiff_w2
          a          b          a          b
2020-01-01    NaN    NaN    NaN    NaN
2020-01-02    NaN    NaN    NaN    NaN
```

```

2020-01-03 -0.500000  0.083333      NaN      NaN
2020-01-04 -0.625000  0.075000 -0.875000  0.175000
2020-01-05  0.011111 -0.183333 -0.100000 -0.100000
2020-01-06  0.166667 -0.416667  0.166667 -0.416667
2020-01-07  0.128571 -0.571429  0.228571 -0.821429

```

Hint

vbt.IF is a shortcut for `IndicatorFactory`.

As you see, `IndicatorFactory` took the specification for our indicator and created an entire Python class that knows how to communicate with `IndicatorBase.run_pipeline` and manipulate and format its results. In particular, it attached the class method `MADiff.run` that looks exactly like `custom_func` but prepares and forwards all arguments to `IndicatorBase.run_pipeline` under the hood. Whenever we call the `run` method, it sets up and returns an instance of `MADiff` with all the input and output data.

You might ask: "Why doesn't the factory create a function instead of a class? Having an indicator function would be more intuitive!" If you read through [Building blocks](#), you would already be familiar with the class `Analyzable`, which is the go-to class for analyzing data. The indicator class created by the factory is a subclass of `Analyzable`, so we not only have access to the output, but also to many methods for analyzing this output! For example, the factory automatically attaches `crossed_above`, `cross_below`, `stats`, and many other methods for each input and output that appears in the indicator:

```

>>> madiff.diff_stats(column=(2, 3, 'a'))
Start      2020-01-01 00:00:00
End        2020-01-07 00:00:00
Period     7 days 00:00:00
Count      5
Mean       -0.16373
Std        0.371153
Min        -0.625
Median     0.011111
Max        0.166667
Min Index  2020-01-04 00:00:00
Max Index  2020-01-06 00:00:00
Name: (2, 3, a), dtype: object

```

Workflow

The main purpose of `IndicatorFactory` is to create a stand-alone indicator class that has a `run` method for running the indicator. For this, it needs to know what inputs, parameters, and outputs to expect. This information can be passed in form of `input_names`, `param_names`, and other arguments to the constructor:

```

>>> MADiff_factory = vbt.IF(
...     class_name='MADiff',
...     input_names=['ts1', 'ts2'],
...     param_names=['w1', 'w2'],
...     output_names=['diff'],
... )
>>> MADiff_factory.Indicator
vectorbtpro.indicators.factory.MADiff

```

Upon the initialization, it creates the skeleton of our indicator class of type `IndicatorBase`, accessible via `IndicatorFactory.Indicator`. Even though the factory has created the constructor of this class and attached various properties and methods for working with it, we can't run the indicator:

```

>>> MADiff_factory.Indicator.run()
NotImplementedError:

```

This is because we haven't provided it with the calculation function yet. To do this, there are multiple methods starting with the prefix `with_`. The base method all other methods are based upon is `IndicatorFactory.with_custom_func` (which we used above) - it overrides the abstract `run` method to execute the indicator using `IndicatorBase.run_pipeline` and returns a ready-to-use indicator class:

```

>>> MADiff = MADiff_factory.with_custom_func(custom_func)
>>> MADiff
vectorbtpro.indicators.factory.MADiff

```

The calculation function has been attached successfully, we can now run this indicator!

Factory methods

Factory methods come in two different flavors: instance and class methods. The instance methods with the prefix `with_`, such as `IndicatorFactory.with_custom_func`, require instantiation of the indicator factory. That is, we have to do `vbt.IF(...)` and provide the required information manually as we did with `MADiff`. The class methods with the prefix `from_`, such as `IndicatorFactory.from_expr`, can parse the required information (semi-)automatically.

From custom function

The method `IndicatorFactory.with_custom_func` takes a so-called "custom function", which is the most flexible way to define an indicator. But with great power comes great responsibility: it's up to the user to iterate through parameters, handle caching, and concatenate columns for each parameter (usually by `apply_and_concat`). Also, we must ensure that each output array has an appropriate number of columns, which is the number of columns in the input arrays multiplied by the number of parameter combinations. Additionally, the custom function receives commands passed by the pipeline, and it's the task of the user to properly execute those commands.

For example, if our custom function needs the index and the columns along with the NumPy arrays, we can instruct the pipeline to pass the wrapper, which is done by setting `pass_wrapper=True` in `with_custom_func`. This as well as all other arguments are forwarded directly to `IndicatorBase.run_pipeline`, which takes care of communicating with our custom function.

From apply function

The method `IndicatorFactory.with_apply_func` simplifies indicator development a lot: it creates `custom_func` that handles caching, iteration over parameters with `apply_and_concat`, output concatenation with `column_stack`, and passes this function to `IndicatorFactory.with_custom_func`. Our part is writing a so-called "apply function", which accepts a single parameter combination and does the calculation. The resulting outputs are automatically concatenated along the column axis.

Note

An apply function has mostly the same signature as a custom function, but the parameters are single values as opposed to multiple values.

Let's implement our indicator using an apply function:

```
>>> def apply_func(ts1, ts2, w1, w2):
...     ts1_ma = vbt.nb.rolling_mean_nb(ts1, w1) / ts1
...     ts2_ma = vbt.nb.rolling_mean_nb(ts2, w2) / ts2
...     return ts1_ma - ts2_ma

>>> MADiff = vbt.IF(
...     class_name='MADiff',
...     input_names=['ts1', 'ts2'],
...     param_names=['w1', 'w2'],
...     output_names=['diff'],
... ).with_apply_func(apply_func)

>>> madiff = MADiff.run(ts1, ts2, w1, w2)
>>> madiff.diff
madiff_w1          3          2
madiff_w2          4          4
              a      b      a      b
2020-01-01      NaN      NaN      NaN      NaN
2020-01-02      NaN      NaN      NaN      NaN
2020-01-03 -0.500000  0.083333      NaN      NaN
2020-01-04 -0.625000  0.075000 -0.875000  0.175000
2020-01-05  0.011111 -0.183333 -0.100000 -0.100000
2020-01-06  0.166667 -0.416667  0.166667 -0.416667
2020-01-07  0.128571 -0.571429  0.228571 -0.821429
```

That's it! Under the hood, our code created a custom function that iterates over both parameter combinations and calls `apply_func` on each one. If we printed `ts1`, `ts2`, `w1`, and `w2`, we would see that `ts1` and `ts2` are the same, while `w1` and `w2` are now single values. This way, we can entirely abstract ourselves from the number of parameter combinations and work with a single set of parameters at a time.

Another advantage of this method is that apply functions are natural inhabitants of vectorbt 🐍 and we can use most regular and Numba-compiled functions that take two-dimensional NumPy arrays directly as apply functions! Let's illustrate this by building an indicator for the rolling covariance:

```
>>> RollCov = vbt.IF(
...     class_name='RollCov',
...     input_names=['ts1', 'ts2'],
...     param_names=['w'],
...     output_names=['rollcov'],
... ).with_apply_func(vbt.nb.rolling_cov_nb)

>>> rollcov = RollCov.run(ts1, ts2, [2, 3])
>>> rollcov.rollcov
rollcov_w          2          3
              a      b      a      b
2020-01-01      NaN      NaN      NaN      NaN
2020-01-02 -0.25  0.25      NaN      NaN
2020-01-03 -0.25  0.25 -0.666667  0.666667
2020-01-04 -0.25  0.25 -0.666667  0.666667
2020-01-05  0.25 -0.25  0.000000  0.000000
2020-01-06  0.25 -0.25  0.666667 -0.666667
2020-01-07  0.25 -0.25  0.666667 -0.666667
```

Here, the both input arrays and the window parameter were passed directly to `rolling_cov_nb`.

Custom iteration

We can easily emulate `apply_func` using `custom_func` and `apply_and_concat`, for example, if we need the index of the current iteration and/or want to have access to all parameter combinations:

```
>>> from vectorbtpro.base.combining import apply_and_concat

>>> def apply_func(i, ts1, ts2, w): ①
...     return vbt.nb.rolling_cov_nb(ts1, ts2, w[i])

>>> def custom_func(ts1, ts2, w):
...     return apply_and_concat(len(w), apply_func, ts1, ts2, w) ②

>>> RollCov = vbt.IF(
...     class_name='RollCov',
...     input_names=['ts1', 'ts2'],
...     param_names=['w'],
...     output_names=['rollcov'],
```

```
... ).with_custom_func(custom_func)
```

- 1 In contrast to our previous `apply_func`, an apply function used in `apply_and_concat` must take the index of the iteration and select the parameters manually using this index
- 2 `apply_and_concat` requires the number of iterations, which is simply the length of any parameter array

The same using `IndicatorFactory.with_apply_func` and `select_params=False`:

```
>>> RollCov = vbt.IF(
...     class_name='RollCov',
...     input_names=['ts1', 'ts2'],
...     param_names=['w'],
...     output_names=['rollcov'],
... ).with_apply_func(apply_func, select_params=False)
```

Execution

Since the same apply function is being called multiple times - once per parameter combination -, we can use one of the vectorbt's preset execution engines to distribute those calls sequentially (default), across multiple threads, or across multiple processes. In fact, the function `apply_and_concat`, which is used to iterate over all parameter combinations, takes care of this automatically by forwarding all calls to the executor function `execute`. Using keyword arguments in `execute_kwargs`, we can define the rules by which to distribute those calls. For example, to disable the progress bar:

```
>>> RollCov = vbt.IF(
...     class_name='RollCov',
...     input_names=['ts1', 'ts2'],
...     param_names=['w'],
...     output_names=['rollcov'],
... ).with_apply_func(vbt.nb.rolling_cov_nb)

>>> RollCov.run(
...     ts1, ts2, np.full(100, 2),
...     execute_kwargs=dict(show_progress=False)
... )
```

Iteration 100/100

Numba

When the apply function is Numba-compiled, the indicator factory makes the parameter selection function Numba-compiled as well (+ with GIL released), so we can utilize multithreading. This entire behavior can be disabled by setting `jit_select_params` to False. The keyword arguments used to set up the Numba-compiled function can be passed via the `jit_kwargs` argument.

Note

Setting `jit_select_params` will remove all keyword arguments since variable keyword arguments aren't supported by Numba (yet). To pass keyword arguments to the apply function anyway, set `remove_kwargs` to False or use the `kwargs_as_args` argument, which specifies which keyword arguments should be supplied as (variable) positional arguments.

Additionally, we can explicitly set `jitted_loop` to True to loop over each parameter combination in a Numba loop, which speeds up the iteration for shallow inputs over a huge number of columns, but slows it down otherwise.

Note

In this case, the execution will be performed by Numba, so you can't use `execute_kwargs` anymore.

Debugging

Sometimes, it's not that clear which arguments are being passed to `apply_func`. Debugging in this scenario is usually easy: just replace your apply function with a generic apply function that takes variables arguments, and print those.

```
>>> def apply_func(*args, **kwargs):
...     for i, arg in enumerate(args):
...         print("arg {}: {}".format(i, type(arg)))
...     for k, v in kwargs.items():
...         print("kwarg {}: {}".format(k, type(v)))
...     raise NotImplementedError

>>> RollCov = vbt.IF(
...     class_name='RollCov',
...     input_names=['ts1', 'ts2'],
...     param_names=['w'],
...     output_names=['rollcov'],
... ).with_apply_func(apply_func, select_params=False)

>>> try:
...     RollCov.run(ts1, ts2, [2, 3], some_arg="some_value")
... except:
...     pass
arg 0: <class 'int'>
arg 1: <class 'numpy.ndarray'>
arg 2: <class 'numpy.ndarray'>
arg 3: <class 'list'>
kwarg some_arg: <class 'str'>
```

From parsing

Parsers are the most convenient way to build indicator classes. For instance, there are dedicated parser methods for third-party technical analysis packages that can derive the specification of each indicator in an (semi-)automated way. In addition, there is a powerful expression parser to avoid writing complex Python functions for simpler indicators. Let's *express* our indicator as an expression:

```
>>> MADiff = vbt.IF.from_expr(
...     "rolling_mean(@in_ts1, @p_w1) / @in_ts1 - rolling_mean(@in_ts2, @p_w2) / @in_ts2",
...     factory_kwargs=dict(class_name="MADiff") ❶
... )
>>> madiff = MADiff.run(ts1, ts2, w1, w2)
>>> madiff.out
madiff_w1          2
madiff_w2          4
              a      b      a      b
2020-01-01      NaN      NaN      NaN      NaN
2020-01-02      NaN      NaN      NaN      NaN
2020-01-03 -0.500000  0.083333      NaN      NaN
2020-01-04 -0.625000  0.075000 -0.875000  0.175000
2020-01-05  0.011111 -0.183333 -0.100000 -0.100000
2020-01-06  0.166667 -0.416667  0.166667 -0.416667
2020-01-07  0.128571 -0.571429  0.228571 -0.821429
```

❶ We can still override any information passed to the factory class

Notice how we didn't have to call `vbt.IF(...)`? `IndicatorFactory.from_expr` is a class method that parses `input_names` and other information from the expression and creates a factory instance using solely this information. Crazy how we compressed our first implementation with `mov_avg_crossover` to just this while enjoying all the perks, right?

Run methods

Once we built our indicator class, it's time to run it. The main method for executing an indicator is the class method `IndicatorBase.run`, which accepts positional and keyword arguments based on the specification provided to the `IndicatorFactory`. These arguments include input arrays, in-place output arrays, and parameters. Any additional arguments are forwarded down to `IndicatorBase.run_pipeline`, which can either use them to set up the pipeline, or forward them further down to the custom function and then, if provided, the apply function.

To see what arguments the `run` method accepts, use `phelp`:

```
>>> vbt.phelp(MADiff.run)
MADiff.run(
    ts1,
    ts2,
    w1,
    w2,
    short_name='madiff',
    hide_params=None,
    hide_default=True,
    **kwargs
):
    Run 'MADiff' indicator.

    * Inputs: 'ts1', 'ts2'
    * Parameters: 'w1', 'w2'
    * Outputs: 'out'

    Pass a list of parameter names as 'hide_params' to hide their column levels.
    Set 'hide_default' to False to show the column levels of the parameters with a default value.

    Other keyword arguments are passed to 'MADiff.run_pipeline'.
```

We see that `MADiff.run` takes two input time series `ts1` and `ts2`, two parameters `w1` and `w2`, and produces a single output time series `diff`. Upon calling the class method, it runs the indicator and returns a new instance of `MADiff` with all the data being ready for analysis. In particular, we can access the output as a regular instance attribute `MADiff.diff`.

The second method for running indicators is `IndicatorBase.run_combs`, which takes the same inputs as the method above, but computes all combinations of the passed parameters based on a combinatorial function and returns **multiple** indicator instances that can be combined with each other. This is useful to compare multiple indicators of the **same type** but different parameters, such as for testing a moving average crossover, which involves two `MA` instances applied on the same time series:

```
>>> ts = pd.Series([3, 2, 1, 2, 3])
>>> fast_ma, slow_ma = vbt.MA.run_combs(
...     ts, [2, 3, 4],
...     short_names=['fast_ma', 'slow_ma'])
>>> fast_ma.ma_crossed_above(slow_ma)
fast_ma_window      2      3
slow_ma_window      3      4      4
0          False  False  False
1          False  False  False
2          False  False  False
3          False  False  False
4           True   True  False
```

In the example above, `MA.run_combs` generated the combinations of `window` using `itertools.combinations` and `r=2`. The first set of window combinations was passed to the first instance, the second set to the second instance. The above example can be easily replicated using the `run` method alone:

```
>>> windows = [2, 3, 4]
>>> fast_windows, slow_windows = zip(*combinations(windows, 2))
>>> fast_ma = vbt.MA.run(ts, fast_windows, short_name='fast_ma')
>>> slow_ma = vbt.MA.run(ts, slow_windows, short_name='slow_ma')
>>> fast_ma.ma_crossed_above(slow_ma)
fast_ma_window      2      3
slow_ma_window      3      4      4
0      False  False  False
1      False  False  False
2      False  False  False
3      False  False  False
4      True   True   False
```

The main advantage of a single `run_combs` call over multiple `run` calls is that it doesn't need to re-compute each combination thanks to smart caching.

Note

`run_combs` should be only used for combining multiple indicators. To test multiple parameter combinations, use `run` and provide parameters as lists.

Preset indicators

VectorBT PRO implements a collection of preset, fully Numba-compiled indicators (such as [ATR](#)) that take advantage of manual caching, extending, and plotting. You can use them to take an inspiration on how to create indicators in a classic but performant way.

Note

vectorbt uses SMA and EMA, while other technical analysis libraries and TradingView use the Wilder's method. There is no right or wrong method. See [different smoothing methods](#).

[Python code](#)

Documentation Indicators

Development

VectorBT PRO implements a ton of functions and arguments for seamless development of indicators. All it takes is an indicator function and a specification of how to handle it.

Parameters

IndicatorFactory allows definition of arbitrary parameter grids. An indicator can have one or more parameters. A parameter can have one or more values. Each value can be a scalar (such as integer), an array, or any other object.

If an indicator has multiple parameters, and one or more of them have multiple values, their values will broadcast against each other. For example, if the parameter `w1` has only one value of `2` and the parameter `w2` has two values of `3` and `4`, then `w1` will be "stretched" to two values: `2` and `2`. This way, the indicator can **zip** both parameters and create two parameter combinations: `(2, 3)` and `(2, 4)`. It will then iterate over the list of those combinations and apply a function on each one. Here's an illustration of broadcasting:

```
>>> from vectorbtpro import *

>>> def broadcast_params(*params):
...     return list(zip(*vbt.broadcast(*[vbt.to_1d_array(p) for p in params])))

>>> broadcast_params(2, 3)
[(2, 3)]

>>> broadcast_params([2, 3], 4)
[(2, 4), (3, 4)]

>>> broadcast_params(2, [3, 4])
[(2, 3), (2, 4)]

>>> broadcast_params([2, 3], [4, 5])
[(2, 4), (3, 5)]

>>> broadcast_params([2, 3], [4, 5, 6])
ValueError: Could not broadcast shapes: {0: (2,), 1: (3,)}
```

Note

You shouldn't confuse a broadcasting operation with a product operation. The product of `[2, 3]` and `[4, 5]` would yield 4 combinations: `[2, 4]`, `[2, 5]`, `[3, 4]`, and `[3, 5]`. The broadcasting operation simply stretches smaller arrays to the length of bigger arrays for zipping purposes.

To illustrate the usage of parameters in indicators, let's build a simplified indicator that returns 1 when the rolling mean is above an upper bound, -1 if it's below a lower bound, and 0 if it's between the upper and the lower bound:

```
>>> def apply_func(ts, window, lower, upper):
...     out = np.full_like(ts, np.nan, dtype=float_)
...     ts_mean = vbt.nb.rolling_mean_nb(ts, window)
...     out[ts_mean >= upper] = 1
...     out[ts_mean <= lower] = -1
...     out[(ts_mean > lower) & (ts_mean < upper)] = 0
...     return out

>>> Bounded = vbt.IF(
...     class_name="Bounded",
...     input_names=['ts'],
...     param_names=['window', 'lower', 'upper'],
...     output_names=['out']
... ).with_apply_func(apply_func)

>>> def generate_index(n):
...     return vbt.date_range("2020-01-01", periods=n)

>>> ts = pd.DataFrame({
...     'a': [5, 4, 3, 2, 3, 4, 5],
...     'b': [2, 3, 4, 5, 4, 3, 2]
... }, index=generate_index(7))
>>> bounded = Bounded.run(ts, 2, 3, 5)
```

To get the list of parameter names:

```
>>> bounded.param_names
('window', 'lower', 'upper')
```

The (broadcasted) values of each parameter can be accessed as an attribute of the indicator instance called by the parameter name plus `_list`:

```
>>> bounded.window_list
[2]
```


By default, when `per_column` is set to `False`, each parameter combination is applied on each column in the input. This means: if our input array has 20 columns and we want to test 5 parameter combinations, we will get $20 * 5 = 100$ columns in total.

One parameter combination:

```
>>> Bounded.run(
...     ts,
...     window=2,
...     lower=3,
...     upper=5
... ).out ①
bounded_window      2
bounded_lower       3
bounded_upper       5
      a  b
2020-01-01      NaN NaN
2020-01-02      0.0 -1.0
2020-01-03      0.0  0.0
2020-01-04     -1.0  0.0
2020-01-05     -1.0  0.0
2020-01-06      0.0  0.0
2020-01-07      0.0 -1.0
```

① $2 * 1 = 2$ output columns

Multiple parameter combinations:

```
>>> Bounded.run(
...     ts,
...     window=[2, 3],
...     lower=3,
...     upper=5
... ).out ①
bounded_window      2      3
bounded_lower       3      3
bounded_upper       5      5
      a  b  a  b
2020-01-01      NaN NaN NaN NaN
2020-01-02      0.0 -1.0 NaN NaN
2020-01-03      0.0  0.0  0.0 -1.0
2020-01-04     -1.0  0.0 -1.0  0.0
2020-01-05     -1.0  0.0 -1.0  0.0
2020-01-06      0.0  0.0 -1.0  0.0
2020-01-07      0.0 -1.0  0.0 -1.0
```

① $2 * 2 = 4$ output columns

Product of parameter combinations:

```
>>> Bounded.run(
...     ts,
...     window=[2, 3],
...     lower=[3, 4],
...     upper=5,
...     param_product=True
... ).out ①
bounded_window      2      3      3      4
bounded_lower       3      4      3      4
bounded_upper       5      5      5      5
      a  b  a  b  a  b  a  b
2020-01-01      NaN NaN NaN NaN NaN NaN NaN NaN
2020-01-02      0.0 -1.0  0.0 -1.0 NaN NaN NaN NaN
2020-01-03      0.0  0.0 -1.0 -1.0  0.0 -1.0 -1.0 -1.0
2020-01-04     -1.0  0.0 -1.0  0.0 -1.0  0.0 -1.0 -1.0
2020-01-05     -1.0  0.0 -1.0  0.0 -1.0  0.0 -1.0  0.0
2020-01-06      0.0  0.0 -1.0 -1.0 -1.0  0.0 -1.0 -1.0
2020-01-07      0.0 -1.0  0.0 -1.0  0.0 -1.0 -1.0 -1.0
```

① $2 * 2 * 2 = 8$ output columns

More exotic parameter combinations can be created using `generate_param_combs`. Since the lower bound should always remain lower than the upper bound, we can account for this relationship using `itertools.combinations`. After that, we can build a Cartesian product with the window using `itertools.product`.

```
>>> bound_combs_op = (combinations, [3, 4, 5], 2)
>>> product_op = (product, [2, 3], bound_combs_op)
>>> windows, lowers, uppers = vbt.generate_param_combs(product_op) ①

>>> Bounded.run(
...     ts,
...     window=windows,
...     lower=lowers,
...     upper=uppers
... ).out ②
bounded_window      2      3      4      3      4
bounded_lower       4      5      5      4      5
bounded_upper       5      5      5      5      5
      a  b  a  b  a  b  a  b  a  b
2020-01-01      NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN
2020-01-02      1.0 -1.0  0.0 -1.0  0.0 -1.0 NaN NaN NaN NaN
2020-01-03      0.0  0.0  0.0  0.0 -1.0 -1.0  1.0 -1.0  0.0 -1.0 -1.0
2020-01-04     -1.0  1.0 -1.0  0.0 -1.0  0.0 -1.0  1.0 -1.0  0.0 -1.0 -1.0
2020-01-05     -1.0  1.0 -1.0  0.0 -1.0  0.0 -1.0  1.0 -1.0  0.0 -1.0  0.0
```

```

2020-01-06    0.0  0.0  0.0  0.0 -1.0 -1.0 -1.0  1.0 -1.0  0.0 -1.0 -1.0
2020-01-07    1.0 -1.0  0.0 -1.0  0.0 -1.0  1.0 -1.0  0.0 -1.0 -1.0 -1.0

```

- 1 Each tuple represents a combinatoric operation. Tuples can take other tuples as inputs.
- 2 $2 * 2 * 2 * 2 * 3 / 2 = 12$ output columns

One parameter combination per column:

```

>>> Bounded.run(
...     ts,
...     window=[2, 3],
...     lower=[3, 4],
...     upper=5,
...     per_column=True
... ).out ①
bounded_window    2    3
bounded_lower     3    4
bounded_upper     5    5
a    b
2020-01-01      NaN NaN
2020-01-02      0.0 NaN
2020-01-03      0.0 -1.0
2020-01-04     -1.0 -1.0
2020-01-05     -1.0  0.0
2020-01-06      0.0 -1.0
2020-01-07      0.0 -1.0

```

- 1 The number of output columns matches the number of input columns

Defaults

Any argument passed to `IndicatorFactory.with_custom_func` that isn't listed among the arguments of `IndicatorBase.run_pipeline` is meant to be used as a default argument for the calculation function. Since most methods, including `IndicatorFactory.with_apply_func`, call this method, we can easily define parameter defaults by passing them along with the function:

```

>>> Bounded = vbt.IF(
...     class_name="Bounded",
...     input_names=['ts'],
...     param_names=['window', 'lower', 'upper'],
...     output_names=['out']
... ).with_apply_func(apply_func, window=2, lower=3, upper=4)

>>> Bounded.run(ts).out
a    b
2020-01-01  NaN NaN
2020-01-02  1.0 -1.0
2020-01-03  0.0  0.0
2020-01-04 -1.0  1.0
2020-01-05 -1.0  1.0
2020-01-06  0.0  0.0
2020-01-07  1.0 -1.0

>>> Bounded.run(ts, upper=[5, 6]).out
bounded_upper    5    6
a    b    a    b
2020-01-01    NaN NaN NaN NaN
2020-01-02    0.0 -1.0 0.0 -1.0
2020-01-03    0.0  0.0 0.0  0.0
2020-01-04   -1.0  0.0 -1.0  0.0
2020-01-05   -1.0  0.0 -1.0  0.0
2020-01-06    0.0  0.0 0.0  0.0
2020-01-07    0.0 -1.0 0.0 -1.0

```

The reason why the parameters `window` and `lower` do not appear in the column hierarchy above is because default values are hidden by default. To uncover them, disable `hide_default`:

```

>>> Bounded.run(ts, hide_default=False).out
bounded_window    2
bounded_lower     3
bounded_upper     4
a    b
2020-01-01      NaN NaN
2020-01-02      1.0 -1.0
2020-01-03      0.0  0.0
2020-01-04     -1.0  1.0
2020-01-05     -1.0  1.0
2020-01-06      0.0  0.0
2020-01-07      1.0 -1.0

```

Array-like

Some parameters are meant to be defined per row, column, or element of the input. By default, if we pass the parameter value as an array, the indicator will treat this array as a list of multiple values - one per input. To make the indicator view this array as a single value, we need to set the flag `is_array_like` to True in `param_settings`. Also, to automatically broadcast the parameter value to the input shape, set `bc_to_input` to True, 0 (index axis), or 1 (column axis).

In our example, the parameter `window` can broadcast per column, and both parameters `lower` and `upper` can broadcast per element. But to make all of this work, we need to rewrite the `apply_func` to apply the rolling mean on each column instead of the entire input:

```

>>> def apply_func(ts, window, lower, upper): ❶
...     out = np.full_like(ts, np.nan, dtype=float_)
...     ts_means = []
...     for col in range(ts.shape[1]):
...         ts_means.append(vbt.nb.rolling_mean_1d_nb(ts[:, col], window[col]))
...     ts_mean = np.column_stack(ts_means)
...     out[ts_mean >= upper] = 1
...     out[ts_mean <= lower] = -1
...     out[(ts_mean > lower) & (ts_mean < upper)] = 0
...     return out

>>> Bounded = vbt.IF(
...     class_name="Bounded",
...     input_names=['ts'],
...     param_names=['window', 'lower', 'upper'],
...     output_names=['out']
... ).with_apply_func(
...     apply_func,
...     param_settings=dict(
...         window=dict(is_array_like=True, bc_to_input=1, per_column=True),
...         lower=dict(is_array_like=True, bc_to_input=True),
...         upper=dict(is_array_like=True, bc_to_input=True)
...     )
... )

```

- ❶ All parameters are now arrays that can broadcast against `ts`

Both bound parameters can now be passed as a scalar (value per whole input), a one-dimensional array (value per row or column, depending upon whether input is a Series or a DataFrame), a two-dimensional array (value per element), or a list of any of those. This allows for the highest parameter flexibility.

For example, let's build a grid of two parameter combinations:

```

>>> Bounded.run(
...     ts,
...     window=np.array([2, 3]), ❶
...     lower=np.array([[1, 2]]), ❷
...     upper=np.array([6, 5, 4, 3, 4, 5, 6]), ❸
... ).out
bounded_window      2      3      4
bounded_lower  array_0 array_0 array_1 array_1
bounded_upper  array_0 array_0 array_1 array_1
                a      b      a      b
2020-01-01      NaN    NaN    NaN    NaN
2020-01-02      0.0    NaN    NaN    NaN
2020-01-03      0.0    0.0    NaN    NaN
2020-01-04      0.0    1.0    1.0    1.0
2020-01-05      0.0    1.0    0.0    1.0
2020-01-06      0.0    0.0    0.0    0.0
2020-01-07      0.0    0.0    0.0    0.0

```

- ❶ One combination with value per column and one with value per input
 ❷ Value per column
 ❸ Value per row

Our `apply_func` gets called twice, one for each parameter combination in `window`. If you print the shapes of the passed arguments, you will see that each window array now matches the number of columns in `ts`, while each bound array exactly matches the shape of `ts`:

```

Combination 1:
(7, 2)
(2,)
(7, 2)
(7, 2)

Combination 2:
(7, 2)
(2,)
(7, 2)
(7, 2)

```

Lazy broadcasting

Broadcasting a huge number of parameters to the input shape can consume lots of memory, especially when the arrays materialize. Luckily, vectorbt can preserve the original (small) dimensions of each parameter array and give us all the power over its broadcasting. This requires setting `keep_flex` to True in `broadcast_kwargs`, which will make the factory first check whether the array can broadcast, and then expand it to either one or two dimensions in the most memory-efficient way. There are two configs in `configs` exactly for this purpose: one for column-wise broadcasting and one for element-wise broadcasting.

```

>>> def apply_func(ts, window, lower, upper):
...     window = np.broadcast_to(window, ts.shape[1]) ❶
...     lower = np.broadcast_to(lower, ts.shape)
...     upper = np.broadcast_to(upper, ts.shape)
...
...     out = np.full_like(ts, np.nan, dtype=float_)
...     ts_means = []
...     for col in range(ts.shape[1]):
...         ts_means.append(vbt.nb.rolling_mean_1d_nb(ts[:, col], window[col]))
...     ts_mean = np.column_stack(ts_means)
...     out[ts_mean >= upper] = 1
...     out[ts_mean <= lower] = -1

```

```

... out[(ts_mean > lower) & (ts_mean < upper)] = 0
... return out

>>> Bounded = vbt.IF(
...     class_name="Bounded",
...     input_names=['ts'],
...     param_names=['window', 'lower', 'upper'],
...     output_names=['out']
... ).with_apply_func(
...     apply_func,
...     param_settings=dict(
...         window=vbt.flex_col_param_config,
...         lower=vbt.flex_elem_param_config,
...         upper=vbt.flex_elem_param_config
...     )
... )

```

1 Bring to the full shape

Well done! This is the most flexible and the least memory consuming indicator implementation. Instead of broadcasting all array-like parameter values right away, we postpone the operation to the point in time when this is actually needed.

With Numba

The implementation above is great but not the most optimized one since it iterates over the input shape multiple times. As a bonus, let's rewrite our `apply_func` to be Numba-compiled: it will iterate over columns and rows, select each parameter value **flexibly** and entirely without broadcasting, and gradually fill the output array.

```

>>> @njit
... def apply_func_nb(ts, window, lower, upper):
...     out = np.full_like(ts, np.nan, dtype=float_)
...
...     for col in range(ts.shape[1]):
...         _window = vbt.flex_select_1d_pc_nb(window, col) 1
...
...         for row in range(ts.shape[0]):
...             window_start = max(0, row + 1 - _window)
...             window_end = row + 1
...             if window_end - window_start >= _window:
...                 _lower = vbt.flex_select_nb(lower, row, col) 2
...                 _upper = vbt.flex_select_nb(upper, row, col) 3
...
...                 mean = np.nanmean(ts[window_start:window_end, col]) 4
...                 if mean >= _upper:
...                     out[row, col] = 1
...                 elif mean <= _lower:
...                     out[row, col] = -1
...                 elif _lower < mean < _upper:
...                     out[row, col] = 0
...
...     return out

```

- 1 Get the window defined for this column
- 2 Get the lower bound defined for this element
- 3 Get the upper bound defined for this element
- 4 Get the mean of this window

Hint

This is a perfectly valid Python code - even if you remove the `@njit` decorator, it would still work!

Remember that executing a code jitted with Numba may provide performance that is magnitudes higher than that offered by Python and even Pandas 🚀

Parameterless

Indicators can also be parameterless, such as **OBV**.

Inputs

IndicatorFactory supports passing none, one, or multiple inputs. If multiple inputs were passed, it tries to broadcast them into a single shape with **broadcast** (see **Broadcasting**).

Remember that in vectorbt each column means a separate backtest. That's why in order to use multiple pieces of information, such as OHLCV, we need to provide them as separate Pandas objects rather than a monolithic DataFrame (see **Multidimensionality**).

Let's create a parameterless indicator that measures the position of the closing price relative to the candle:

```

>>> def apply_func(high, low, close):
...     return (close - low) / (high - low)

>>> RelClose = vbt.IF(
...     input_names=['high', 'low', 'close'],
...     output_names=['out']
... ).with_apply_func(apply_func)

```

```
>>> close = pd.Series([1, 2, 3, 4, 5], index=generate_index(5))
>>> high = close * 1.2
>>> low = close * 0.8

>>> rel_close = RelClose.run(high, low, close)
>>> rel_close.out
2020-01-01    0.5
2020-01-02    0.5
2020-01-03    0.5
2020-01-04    0.5
2020-01-05    0.5
dtype: float64
```

To get the list of input names:

```
>>> rel_close.input_names
('high', 'low', 'close')
```

Any (broadcasted and tiled) input array can be accessed as an attribute of the indicator instance:

```
>>> rel_close.high
2020-01-01    1.2
2020-01-02    2.4
2020-01-03    3.6
2020-01-04    4.8
2020-01-05    6.0
dtype: float64
```

Note

The input array attached to the indicator instance may not look the same as the input passed to the indicator: 1) it was broadcasted with another inputs, and 2) upon accessing the attribute, it gets automatically tiled by the number of parameter combinations to compare it more easily with outputs. To access the original array, prepend an underscore (`_high`).

To demonstrate broadcasting, let's pass `high` as a scalar, `low` as a Series, and `close` as a DataFrame (even if it doesn't make sense):

```
>>> high = 10
>>> low = pd.Series([1, 2, 3, 4, 5], index=generate_index(5))
>>> close = pd.DataFrame({
...     'a': [3, 2, 1, 2, 3],
...     'b': [5, 4, 3, 4, 5]
... }, index=generate_index(5))
>>> RelClose.run(high, low, close).out
              a      b
2020-01-01  0.222222  0.444444
2020-01-02  0.000000  0.250000
2020-01-03 -0.285714  0.000000
2020-01-04 -0.333333  0.000000
2020-01-05 -0.400000  0.000000
```

Hint

By default, if all inputs are Series, they are automatically expanded into two-dimensional NumPy arrays. This is done to provide a single array interface since most vectorbt functions primarily work on two-dimensional data. To keep their original dimensions, set `to_2d` to False in `IndicatorFactory.with_apply_func` or any other factory method.

To change any broadcasting rule, we can pass a dict called `broadcast_kwargs`, which gets unfolded and forwarded down to `broadcast`. For example, let's instruct the broadcaster to cast all three arrays into `np.float16`:

```
>>> RelClose.run(
...     high, low, close,
...     broadcast_kwargs=dict(require_kwargs=dict(dtype=np.float16))
... ).out.dtypes
a      float16
b      float16
dtype: object
```

Hint

Remember that any additional keyword arguments passed to a `run` method are forwarded down to `IndicatorBase.run_pipeline`. Thus, we can set up the pipeline during both the creation of the indicator and its execution.

Since all arrays are directly passed to `broadcast`, another possibility is to wrap any of them using a special class `BCO` to override the broadcasting rules just for this particular array:

```
>>> RelClose.run(
...     vbt.BCO(high, require_kwargs=dict(dtype=np.float16)),
...     vbt.BCO(low, require_kwargs=dict(dtype=np.float16)),
...     vbt.BCO(close, require_kwargs=dict(dtype=np.float16))
... ).out.dtypes
a      float16
b      float16
dtype: object
```

One dim

Not always we can (easily) adopt our indicator function to work on two-dimensional data. For instance, to make use of a TA-Lib indicator in `apply_func`, we can pass to it only one column at a time. To instruct `IndicatorFactory.with_apply_func` to split any input and in-output (Pandas or NumPy) array by column, we can use the `takes_1d` argument:

```
>>> import talib

>>> def apply_func_1d(close, timeperiod):
...     return talib.SMA(close.astype(np.double), timeperiod)

>>> SMA = vbt.IF(
...     input_names=['ts'],
...     param_names=['timeperiod'],
...     output_names=['sma']
... ).with_apply_func(apply_func_1d, takes_1d=True)

>>> sma = SMA.run(ts, [3, 4])
>>> sma.sma
custom_timeperiod      a      3      4
                        b      a      b
2020-01-01      NaN      NaN  NaN  NaN
2020-01-02      NaN      NaN  NaN  NaN
2020-01-03      4.000000  3.000000  NaN  NaN
2020-01-04      3.000000  4.000000  3.5  3.5
2020-01-05      2.666667  4.333333  3.0  4.0
2020-01-06      3.000000  4.000000  3.0  4.0
2020-01-07      4.000000  3.000000  3.5  3.5
```

Note

Not to be confused with `per_column`, which also splits by column but applies one parameter combination on one column instead of all columns.

Defaults

Similar to parameters, we can also define defaults for inputs:

```
>>> RelClose = vbt.IF(
...     input_names=['high', 'low', 'close'],
...     output_names=['out']
... ).with_apply_func(
...     apply_func,
...     high=0,
...     low=10
... )

>>> RelClose.run(close).out
      a      b
2020-01-01  0.7  0.5
2020-01-02  0.8  0.6
2020-01-03  0.9  0.7
2020-01-04  0.8  0.6
2020-01-05  0.7  0.5
```

But in contrast to parameters, setting inputs to scalars is often not the best idea. Rather, we want to be able to set them to other inputs, which is possible using [Ref](#):

```
>>> RelClose = vbt.IF(
...     input_names=['high', 'low', 'close'],
...     output_names=['out']
... ).with_apply_func(
...     apply_func,
...     high=vbt.Ref('close'),
...     low=vbt.Ref('close')
... )

>>> RelClose.run(high=high, close=close).out
      a      b
2020-01-01  0.0  0.0
2020-01-02  0.0  0.0
2020-01-03  0.0  0.0
2020-01-04  0.0  0.0
2020-01-05  0.0  0.0
```

Using Pandas

Not always working solely with NumPy arrays is the best approach: sometimes we want to take advantage of the metadata, Pandas, or vectorbt's Pandas extensions. To avoid conversion of Pandas objects to NumPy arrays, we can set `keep_pd` to True.

As an example, let's create an indicator that takes a DataFrame and normalizes it against the mean of each group of columns. The most interesting part of this: the `group_by` for grouping columns will become a parameter!

```
>>> def apply_func(ts, group_by):
...     return ts.vbt.demean(group_by=group_by)

>>> Demeaner = vbt.IF(
...     input_names=['ts'],
...     param_names=['group_by'],
...     output_names=['out']
... )
```

```
... ).with_apply_func(apply_func, keep_pd=True)

>>> ts_wide = pd.DataFrame({
...     'a': [1, 2, 3, 4, 5],
...     'b': [5, 4, 3, 2, 1],
...     'c': [3, 2, 1, 2, 3],
...     'd': [1, 2, 3, 2, 1]
... }, index=generate_index(5))
>>> demeaner = Demeaner.run(ts_wide, group_by=[(0, 0, 1, 1), True])
>>> demeaner.out
custom_group_by      tuple_0      True
               a      b      c      d      a      b      c      d
2020-01-01      -2.0  2.0  1.0 -1.0 -1.5  2.5  0.5 -1.5
2020-01-02      -1.0  1.0  0.0  0.0 -0.5  1.5 -0.5 -0.5
2020-01-03       0.0  0.0 -1.0  1.0  0.5  0.5 -1.5  0.5
2020-01-04       1.0 -1.0  0.0  0.0  1.5 -0.5 -0.5 -0.5
2020-01-05       2.0 -2.0  1.0 -1.0  2.5 -1.5  0.5 -1.5
```

Instead of working on Pandas objects, we can instruct [IndicatorBase.run_pipeline](#) to pass inputs as NumPy arrays along with a [wrapper](#) containing the Pandas metadata:

```
>>> def apply_func(ts, group_by, wrapper): ❶
...     group_map = wrapper.grouper.get_group_map(group_by=group_by)
...     return vbt.nb.demean_nb(ts, group_map)

>>> Demeaner = vbt.IF(
...     input_names=['ts'],
...     param_names=['group_by'],
...     output_names=['out']
... ).with_apply_func(apply_func, pass_wrapper=True)
```

❶ `ts` is a two-dimensional NumPy array, which can be used in most Numba-compiled functions defined across vectorbt

Inputless

What if an indicator doesn't take any input arrays? For instance, we may want to create an indicator that takes an input shape, creates one or more output arrays of this shape, and fills them using information supplied as additional arguments. For this, we can force the user to provide an input shape using

```
require_input_shape.
```

Let's define a generator that emulates random returns and generates a synthetic price, which is a parametrized way of implementing [RandomData](#):

```
>>> def apply_func(input_shape, start, mean, std):
...     rand_returns = np.random.normal(mean, std, input_shape)
...     return start * np.cumprod(1 + rand_returns, axis=0)

>>> RandPrice = vbt.IF(
...     class_name="RandPrice",
...     param_names=['start', 'mean', 'std'],
...     output_names=['out']
... ).with_apply_func(
...     apply_func,
...     require_input_shape=True,
...     start=100, ❶
...     mean=0, ❷
...     std=0.01, ❸
...     seed=42 ❹
... )

>>> RandPrice.run((5, 2)).out
      0      1
0 100.496714 99.861736
1 101.147620 101.382660
2 100.910779 101.145285
3 102.504375 101.921510
4 102.023143 102.474495
```

- ❶ Default starting value
- ❷ Default mean ("centre") of the distribution
- ❸ Default standard deviation (spread or "width") of the distribution
- ❹ [IndicatorBase.run_pipeline](#) supports random seeds

Info

Whenever `require_input_shape` is True, [IndicatorFactory](#) prepends an `input_shape` argument to the `run` method's signature. Without this argument, the `apply_func` itself must decide on the input shape.

But as you see, having integer columns and index is not quite convenient. Gladly, vectorbt allows us to pass `input_index` and `input_columns` !

```
>>> RandPrice.run(
...     (5, 2),
...     input_index=generate_index(5),
...     input_columns=['a', 'b'],
...     mean=[-0.1, 0.1]
... ).out
randprice_mean      -0.1      0.1
               a      b      a      b
2020-01-01  90.496714 89.861736 109.536582 109.534270
```

```

2020-01-02    82.033180    82.244183    120.755278    118.392000
2020-01-03    73.637778    73.827201    130.747876    129.565496
2020-01-04    67.436898    67.011056    142.498409    142.929202
2020-01-05    60.376609    60.673526    155.454330    155.203528

```

One can even build an indicator that decides on the output shape dynamically. Let's create a crazy indicator that spits out an array with a random shape:

```

>>> def custom_func(min_rows=1, max_rows=5, min_cols=1, max_cols=3): ❶
...     n_rows = np.random.randint(min_rows, max_rows)
...     n_cols = np.random.randint(min_cols, max_cols)
...     return np.random.uniform(size=(n_rows, n_cols))

>>> RandShaped = vbt.IF(
...     output_names=['out']
... ).with_custom_func(custom_func)

>>> RandShaped.run(seed=42).out
      0      1
0  0.950714  0.731994
1  0.598658  0.156019
2  0.155995  0.058084

>>> RandShaped.run(seed=43).out
      0
0  0.609067
dtype: float64

>>> RandShaped.run(seed=44).out
      0      1
0  0.104796  0.74464

```

❶ We use `custom_func` instead of `apply_func` since this indicator cannot be parametrized and thus there is no iteration taking place

Outputs

IndicatorFactory supports returning one or multiple outputs. There are two types of outputs: regular and in-place outputs (also called "in-outputs").

Regular

Regular outputs are arrays explicitly returned by the calculation function. Each output must have an exact same shape and match the number of columns in the input shape multiplied by the number of parameter combinations (we should only take care of this requirement when using `custom_func`, while `apply_func` does the tiling job for us). If there is only one output, an array must be returned. If there are multiple outputs, a tuple of multiple arrays must be returned.

Let's demonstrate multiple regular outputs by computing and returning the entries and the exits from a moving average crossover:

```

>>> def apply_func(ts, fastw, sloww, minp=None):
...     fast_ma = vbt.nb.rolling_mean_nb(ts, fastw, minp=minp)
...     slow_ma = vbt.nb.rolling_mean_nb(ts, sloww, minp=minp)
...     entries = vbt.nb.crossed_above_nb(fast_ma, slow_ma)
...     exits = vbt.nb.crossed_above_nb(slow_ma, fast_ma)
...     return (fast_ma, slow_ma, entries, exits) ❶

>>> CrossSig = vbt.IF(
...     class_name="CrossSig",
...     input_names=['ts'],
...     param_names=['fastw', 'sloww'],
...     output_names=['fast_ma', 'slow_ma', 'entries', 'exits']
... ).with_apply_func(apply_func)

>>> ts2 = pd.DataFrame({
...     'a': [1, 2, 3, 2, 1, 2, 3],
...     'b': [3, 2, 1, 2, 3, 2, 1]
... }, index=generate_index(7))
>>> cross_sig = CrossSig.run(ts2, 2, 4)

```

❶ Also return the fast and the slow moving average for analysis

❷ Important

Any output registered in `output_names` must be of the same shape as the broadcasted inputs. This requirement makes possible indexing the indicator instance.

To get the list of output names:

```

>>> cross_sig.output_names
('fast_ma', 'slow_ma', 'entries', 'exits')

```

Any (broadcasted and tiled) output array can be accessed as an attribute of the indicator instance:

```

>>> cross_sig.entries
crosssig_fastw      2      2
crosssig_sloww      4      4
                   a      b
2020-01-01      False False
2020-01-02      False False
2020-01-03      False False
2020-01-04      False False
2020-01-05      False  True

```



```
2020-01-06    False  False
2020-01-07     True   False
```

In-place

In-place outputs are arrays that are not returned but modified in-place. They act as regular inputs when entering the pipeline and as regular outputs when exiting it. In particular:

1. They broadcast together with regular inputs if provided, otherwise an empty array is created
2. They get tiled by the number of hyperparameter combinations
3. Each tile gets modified in-place (**not** returned)
4. After calculation, all the tiles get concatenated to form an output

By default, in-place outputs are created as empty arrays with uninitialized floating values. This allows creation of optional outputs that, if not written, do not occupy much memory. Since not all outputs are meant to be of data type `float`, we can pass `dtype` in the `in_output_settings`.

Let's modify the indicator above by converting both signal arrays to in-outputs:

```
>>> def apply_func(ts, entries, exits, fastw, sloww, minp=None):
...     fast_ma = vbt.nb.rolling_mean_nb(ts, fastw, minp=minp)
...     slow_ma = vbt.nb.rolling_mean_nb(ts, sloww, minp=minp)
...     entries[:] = vbt.nb.crossed_above_nb(fast_ma, slow_ma) ❶
...     exits[:] = vbt.nb.crossed_above_nb(slow_ma, fast_ma)
...     return (fast_ma, slow_ma) ❷

>>> CrossSig = vbt.IF(
...     class_name="CrossSig",
...     input_names=['ts'],
...     in_output_names=['entries', 'exits'],
...     param_names=['fastw', 'sloww'],
...     output_names=['fast_ma', 'slow_ma']
... ).with_apply_func(
...     apply_func,
...     in_output_settings=dict(
...         entries=dict(dtype=np.bool_), ❸
...         exits=dict(dtype=np.bool_)
...     )
... )
>>> cross_sig = CrossSig.run(ts2, 2, 4)
```

- ❶ Both arrays are modified in-place
- ❷ Both arrays aren't returned anymore
- ❸ Instruct `IndicatorBase.run_pipeline` to initialize both signal arrays with the boolean data type

If we print the `output_names`, we will notice that `entries` and `exits` are not there anymore:

```
>>> cross_sig.output_names
('fast_ma', 'slow_ma')
```

To see all in-output arrays, we need to query the `in_output_names` attribute instead:

```
>>> cross_sig.in_output_names
('entries', 'exits')
```

Both signal arrays can be accessed as usual:

```
>>> cross_sig.entries
crosssig_fastw      2
crosssig_sloww      4
               a    b
2020-01-01    False  False
2020-01-02    False  False
2020-01-03    False  False
2020-01-04    False  False
2020-01-05    False   True
2020-01-06    False  False
2020-01-07     True  False
```

Hint

An interesting scenario emerges when there are no regular outputs, only in-outputs. In such case, you should set `output_names` to an empty list, modify all arrays in-place, and return `None`. See the example below.

Defaults

You may ask: "Why should we bother using in-outputs when we can just return regular outputs?" Because we can provide custom data and overwrite it without consuming additional memory. Consider the following example where we keep the first `n` signals in a boolean time series:

```
>>> @njit
... def apply_func_nb(signals, n):
...     for col in range(signals.shape[1]):
...         n_found = 0
```

```

...     for row in range(signals.shape[0]):
...         if signals[row, col]:
...             if n_found >= n:
...                 signals[row, col] = False
...             else:
...                 n_found += 1
...
>>> FirstNSig = vbt.IF(
...     class_name="FirstNSig",
...     in_output_names=['signals'],
...     param_names=['n']
... ).with_apply_func(apply_func_nb)

>>> signals = pd.Series([False, True, True, True, False])
>>> first_n_sig = FirstNSig.run([1, 2, 3], signals=signals)
>>> first_n_sig.signals
firstnsig_n      1      2      3
0         False  False  False
1          True   True   True
2         False   True   True
3         False  False   True
4         False  False  False

```

As you see, one array did the job of two, and this without touching the passed `signals` array!

```

>>> signals
0    False
1     True
2     True
3     True
4    False
dtype: bool

```

Note

In contrast to regular inputs, none of the in-outputs is required when running an indicator, thus they appear in the signature of the `run` method as keyword arguments with `None` as default. Make sure to pass each in-output as a keyword argument after other positional arguments (such as inputs and parameters).

Extra

Any additional output returned by `custom_func` that is not registered in `output_names` is returned in a raw format along with the indicator instance. Such outputs can include objects of any type, especially arrays that have a shape different from that of the inputs. They are not included in the indicator instance simply because the indicator factory doesn't know how to wrap, index, and analyze them, only the user knows. For example, let's return the rolling mean along with its maximum in each column:

```

>>> def custom_func(ts, window): ❶
...     ts_mas = []
...     ts_ma_maxs = []
...     for w in window:
...         ts_ma = vbt.nb.rolling_mean_nb(ts, w)
...         ts_mas.append(ts_ma)
...         ts_ma_maxs.append(np.nanmax(ts_ma, axis=0))
...     return np.column_stack(ts_mas), np.concatenate(ts_ma_maxs)

>>> MAMax = vbt.IF(
...     class_name='MAMax',
...     input_names=['ts'],
...     param_names=['window'],
...     output_names=['ma'],
... ).with_custom_func(custom_func)

>>> ma_ind, ma_max = MAMax.run(ts2, [2, 3]) ❷
>>> ma_ind
mamax_window      2      3
a      b      a      b
2020-01-01    NaN  NaN    NaN    NaN
2020-01-02    1.5  2.5    NaN    NaN
2020-01-03    2.5  1.5    2.000000  2.000000
2020-01-04    2.5  1.5    2.333333  1.666667
2020-01-05    1.5  2.5    2.000000  2.000000
2020-01-06    1.5  2.5    1.666667  2.333333
2020-01-07    2.5  1.5    2.000000  2.000000

>>> ma_ind.wrapper.wrap_reduced(ma_max) ❸
mamax_window
2      a      2.500000
      b      2.500000
3      a      2.333333
      b      2.333333
dtype: float64

```

- ❶ `apply_func` doesn't support extra outputs, only `custom_func`
- ❷ Extra outputs are returned along with the indicator instance
- ❸ Wrap the reduced array using the wrapper of the indicator instance

Lazy

Use `lazy_outputs` argument when constructing an indicator to define lazy outputs - outputs that are computed from "normal" outputs and when explicitly requested. They are available as regular cacheable properties of the indicator instance and can have an arbitrary type. Continuing with the previous example, let's attach a cached property that returns the maximum of the rolling mean:

```
>>> MAMax = vbt.IF(
...     class_name='MAMax',
...     input_names=['ts'],
...     param_names=['window'],
...     output_names=['ma'],
...     lazy_outputs=dict(
...         ma_max=vbt.cached_property(lambda self: self.ma.max())
...     )
... ).with_apply_func(vbt.nb.rolling_mean_nb)

>>> ma_ind = MAMax.run(ts2, [2, 3])
>>> ma_ind.ma_max
mamax_window
2          a    2.500000
          b    2.500000
3          a    2.333333
          b    2.333333
dtype: float64
```

Hint

You can achieve the same result by subclassing `MAMax` and defining the property in the subclass.

Custom arguments

Sometimes, we need to pass arguments that act neither as inputs, in-outputs, or parameters.

Optional

If you look at `apply_func` of `CrossSig`, we take another optional argument `minp`, which regulates the minimal number of observations in a window required to have a value - listing a keyword argument with its default in `custom_func` or `apply_func` is the first way to provide a default value. Another way is to make the argument positional and to provide its default to `IndicatorFactory.with_custom_func` or any other factory method. The default can also be set during the execution in the `run` method.

Variable

Variable arguments, mostly appearing as `*args`, are used to take a *variable* number of arguments. To enable variable arguments, we need to set `var_args` to `True`. The reason for this is the following: when `IndicatorFactory.with_custom_func` builds the `run` method, it needs to reorganize the arguments such that required arguments come before optional arguments. Without the `var_args` flag, the `run` method doesn't expect any additional positional arguments to be passed, which either leads to an error or, more badly, a corrupted result.

Let's add a variable number of inputs:

```
>>> def custom_func(*args):
...     out = None
...     for arr in args:
...         if out is None:
...             out = arr
...         else:
...             out += arr
...     return out

>>> VarArgAdder = vbt.IF(
...     output_names=['out']
... ).with_custom_func(custom_func, var_args=True)

>>> VarArgAdder.run(
...     pd.Series([1, 2, 3]),
...     pd.Series([10, 20, 30]),
...     pd.Series([100, 200, 300])
... ).out
0    111
1    222
2    333
dtype: int64
```

1 No inputs

Note

The indicator above is effectively inputless: inputs that are not registered in `input_names` won't broadcast automatically and are not available as attributes of an indicator instance.

Positional

Positional arguments are treated like variable arguments.

Keyword-only

We can set `keyword_only_args` to `True` to force ourselves to use any argument as a keyword-only argument, for instance, to avoid accidentally misplacing arguments. Take the `RelClose` indicator as an example:

```
>>> def apply_func(high, low, close):
...     return (close - low) / (high - low)

>>> RelClose = vbt.IF(
...     input_names=['high', 'low', 'close'],
...     output_names=['out']
... ).with_apply_func(apply_func)

>>> RelClose.run(close, high, low).out ❶
      a      b
2020-01-01  1.285714  1.800000
2020-01-02  1.000000  1.333333
2020-01-03  0.777778  1.000000
2020-01-04  0.750000  1.000000
2020-01-05  0.714286  1.000000

>>> RelClose = vbt.IF(
...     input_names=['high', 'low', 'close'],
...     output_names=['out']
... ).with_apply_func(apply_func, keyword_only_args=True)

>>> RelClose.run(close, high, low).out ❷
TypeError: run() takes 1 positional argument but 4 were given

>>> RelClose.run(close=close, high=high, low=low).out ❸
      a      b
2020-01-01  0.222222  0.444444
2020-01-02  0.000000  0.250000
2020-01-03 -0.285714  0.000000
2020-01-04 -0.333333  0.000000
2020-01-05 -0.400000  0.000000
```

- ❶ `close` is wrongly passed as the highest price - no error!
- ❷ An error is shown, meaning we have to use keyword arguments
- ❸ `close` is now correctly passed as the closing price

Built-in caching

IndicatorFactory re-uses calculation artifacts whenever possible. Since it was originally designed for hyperparameter optimization and there are times when parameter combinations get repeated, prevention of processing the same parameter combination over and over again is inevitable for good performance.

First, let's take a look at a typical raw output by passing repeating parameter combinations and setting `return_raw` to `True`:

```
>>> raw = vbt.MA.run(
...     ts2,
...     window=[2, 2, 3],
...     wtype=["simple", "simple", "exp"], ❶
...     return_raw=True)
>>> raw
(array([[ nan,      nan,      nan,      nan,      nan,      nan],
       [1.5,    2.5,    1.5,    2.5,      nan,      nan],
       [2.5,    1.5,    2.5,    1.5,    2.25,    1.75],
       [2.5,    1.5,    2.5,    1.5,    2.125,    1.875],
       [1.5,    2.5,    1.5,    2.5,    1.5625,  2.4375],
       [1.5,    2.5,    1.5,    2.5,    1.78125,  2.21875],
       [2.5,    1.5,    2.5,    1.5,    2.390625, 1.609375]]),
 [(2, 0), (2, 0), (3, 2)],
 2,
 [])
```

- ❶ Three parameter combinations with two of them being identical

The raw output consists of

1. the list of returned output arrays,
2. the list of zipped parameter combinations,
3. the number of input columns, and
4. other objects returned along with output arrays but not listed in `output_names`.

Info

A raw output represents the context of running an indicator. If any parameter combination appears in the list of zipped parameter combinations, it means that it was actually run, not cached.

We see that our calculation function was executed for the same parameter combination twice. There is nothing wrong with this if our calculation is fast enough for us to not care about re-running the same calculation procedure. But what if our indicator was very complex and slow to compute? In such case, we can instruct **IndicatorBase.run_pipeline** to run the indicator on unique parameter combinations only by passing `run_unique`:

```
>>> raw = vbt.MA.run(
```

```

...     ts2,
...     window=[2, 2, 3],
...     wtype=["simple", "simple", "exp"],
...     return_raw=True,
...     run_unique=True,
...     silence_warnings=True) ❶
>>> raw
(array([[ nan,    nan,    nan,    nan],
       [1.5,   2.5,    nan,    nan],
       [2.5,   1.5,   2.25,   1.75 ],
       [2.5,   1.5,   2.125,   1.875 ],
       [1.5,   2.5,   1.5625,  2.4375 ],
       [1.5,   2.5,   1.78125,  2.21875 ],
       [2.5,   1.5,   2.390625, 1.609375]]),
 [(2, 0), (3, 2)],
 2,
 [])

```

❶ Without `silence_warnings`, we would get a warning that the raw output contains only unique parameter combinations. You can ignore this.

Let's compare the performance of repeatedly running the same parameter combination with and without `run_unique`:

```

>>> a = np.random.uniform(size=(1000,))

>>> %timeit vbt.MA.run(a, np.full(1000, 2), run_unique=False)
11.6 ms ± 1.26 ms per loop (mean ± std. dev. of 7 runs, 100 loops each)

>>> %timeit vbt.MA.run(a, np.full(1000, 2), run_unique=True)
5.91 ms ± 220 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)

```

Hint

Moving average is among the fastest indicators out there. Try this example on a more complex indicator to get a feeling of how important the built-in caching is!

As a rule of thumb:

- Enable `run_unique` if input arrays have few columns, the calculation function is rather slow, and there are duplicates among parameter combinations
- Disable `run_unique` if input arrays have many columns, the calculation function is very fast, or if two identical parameter combinations can lead to different results (for example, when using `custom_func` that does some decision based on the whole parameter grid, or when there is some randomness involved)

Note

`run_unique` is disabled by default.

Reusing cache

Internally, `run_unique` uses the raw output computed from the unique parameter combinations to produce the output for all parameter combinations. But what if we had our own raw output? We can pass it as `use_raw`! This won't call the calculation function but simply stack raw outputs in the way their parameter combinations appear in the requested grid. If some requested parameter combinations cannot be found in `use_raw`, it will throw an error:

```

>>> raw = vbt.MA.run(
...     ts2,
...     window=[2, 3],
...     wtype=["simple", "exp"],
...     return_raw=True)
>>> vbt.MA.run(ts2, 2, "simple", use_raw=raw).ma
ma_window      2
ma_wtype      simple
      a      b
2020-01-01  NaN  NaN
2020-01-02   1.5  2.5
2020-01-03   2.5  1.5
2020-01-04   2.5  1.5
2020-01-05   1.5  2.5
2020-01-06   1.5  2.5
2020-01-07   2.5  1.5

>>> vbt.MA.run(ts2, 2, "exp", use_raw=raw).ma
ValueError: (2, 2) is not in list

```

This way, we can pre-compute indicators.

Manual caching

Another performance enhancement can be introduced by caching manually, which must be implemented inside `custom_func`. Additionally, `custom_func` must accept a `return_cache` argument for returning the cache and a `use_cache` argument for reusing the cache (similar to `return_raw` and `use_raw`, remember?). Luckily for us, [IndicatorFactory.with_apply_func](#) takes a `cache_func` and implements a `custom_func` that meets the requirements above.

Consider the following scenario: we want to calculate the relative distance between two computationally-expensive rolling windows. We have already decided on the value for the first window, and want to test thousands of values for the second window. Without caching, and even with `run_unique` enabled, the first rolling window will be re-calculated over and over again and waste our resources:

```

>>> def roll_mean_expensive_nb(ts, w):

```

```

...     for i in range(100):
...         out = vbt.nb.rolling_mean_nb(ts, w)
...     return out

>>> def apply_func(ts, w1, w2):
...     roll_mean1 = roll_mean_expensive_nb(ts, w1)
...     roll_mean2 = roll_mean_expensive_nb(ts, w2)
...     return (roll_mean2 - roll_mean1) / roll_mean1

>>> RelMADist = vbt.IF(
...     class_name="RelMADist",
...     input_names=['ts'],
...     param_names=['w1', 'w2'],
...     output_names=['out'],
... ).with_apply_func(apply_func)

>>> RelMADist.run(ts2, 2, 3).out
relmadist_w1      2
relmadist_w2      3
               a      b
2020-01-01      NaN    NaN
2020-01-02      NaN    NaN
2020-01-03    -0.200000  0.333333
2020-01-04    -0.066667  0.111111
2020-01-05    0.333333 -0.200000
2020-01-06    0.111111 -0.066667
2020-01-07    -0.200000  0.333333

>>> %timeit RelMADist.run(ts2, 2, np.arange(2, 1000))
294 ms ± 52.9 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)

```

To avoid this, let's pre-compute all unique rolling windows in `cache_func` and use them in `apply_func`:

```

>>> def cache_func(ts, w1, w2): ①
...     cache_dict = dict()
...     for w in w1 + w2:
...         if w not in cache_dict:
...             cache_dict[w] = roll_mean_expensive_nb(ts, w)
...     return cache_dict

>>> def apply_func(ts, w1, w2, cache_dict): ②
...     return (cache_dict[w2] - cache_dict[w1]) / cache_dict[w1]

>>> RelMADist = vbt.IF(
...     class_name="RelMADist",
...     input_names=['ts'],
...     param_names=['w1', 'w2'],
...     output_names=['out'],
... ).with_apply_func(apply_func, cache_func=cache_func)

>>> RelMADist.run(ts2, 2, 3).out
relmadist_w1      2
relmadist_w2      3
               a      b
2020-01-01      NaN    NaN
2020-01-02      NaN    NaN
2020-01-03    -0.200000  0.333333
2020-01-04    -0.066667  0.111111
2020-01-05    0.333333 -0.200000
2020-01-06    0.111111 -0.066667
2020-01-07    -0.200000  0.333333

>>> %timeit RelMADist.run(ts2, 2, np.arange(2, 1000))
119 ms ± 335 µs per loop (mean ± std. dev. of 7 runs, 10 loops each)

```

① `cache_func` accepts the same arguments as `apply_func` but parameters are now lists instead of single values

② `apply_func` accepts the output of `cache_func` as the last argument. If there are multiple outputs, they all must appear as separate arguments.

We have cut down the processing time in half!

Per column

What happens when the user passes `per_column=True` to apply each parameter combination per column? Internally, `IndicatorFactory.with_apply_func` splits any input, in-output, and parameter array per column, and passes one element of each to `apply_func` at a time. But the same splitting procedure cannot be performed for `cache_func` since we would suddenly get 1) a list of input arrays instead of a single array (if the caching function was Numba-compiled, this would yield an error since Numba doesn't allow the same argument with two different types), and 2) each input array in that list would be different, so keeping a single caching dict with parameter combinations as keys would be not enough.

To account for this edge case, vectorbt passes input and in-output arrays in their regular shape (not split), but it also passes an argument `per_column` set to `True`, such that `cache_func` knows that each parameter corresponds to only one column in the input data. In the caching function, we can then use this flag to decide what to do next. Usually, we just disable caching and calculate everything directly in the apply function.

```

>>> def cache_func(ts, w1, w2, per_column=False):
...     if per_column:
...         return None
...     cache_dict = dict()
...     for w in w1 + w2:
...         if w not in cache_dict:
...             cache_dict[w] = roll_mean_expensive_nb(ts, w)
...     return cache_dict

```

```

>>> def apply_func(ts, w1, w2, cache_dict=None): ❶
...     if cache_dict is None:
...         roll_mean1 = roll_mean_expensive_nb(ts, w1)
...         roll_mean2 = roll_mean_expensive_nb(ts, w2)
...     else:
...         roll_mean1 = cache_dict[w1]
...         roll_mean2 = cache_dict[w2]
...     return (roll_mean2 - roll_mean1) / roll_mean1
...

>>> RelMADist = vbt.IF(
...     class_name="RelMADist",
...     input_names=['ts'],
...     param_names=['w1', 'w2'],
...     output_names=['out'],
... ).with_apply_func(apply_func, cache_func=cache_func)

>>> RelMADist.run(ts2, 2, 3).out
relmadist_w1      2
relmadist_w2      3
               a      b
2020-01-01      NaN    NaN
2020-01-02      NaN    NaN
2020-01-03    -0.200000  0.333333
2020-01-04    -0.066667  0.111111
2020-01-05     0.333333 -0.200000
2020-01-06     0.111111 -0.066667
2020-01-07    -0.200000  0.333333

>>> RelMADist.run(ts2, [2, 2], [3, 4], per_column=True).out
relmadist_w1      2
relmadist_w2      3      4
               a      b
2020-01-01      NaN    NaN
2020-01-02      NaN    NaN
2020-01-03    -0.200000  NaN
2020-01-04    -0.066667  0.333333
2020-01-05     0.333333 -0.200000
2020-01-06     0.111111 -0.200000
2020-01-07    -0.200000  0.333333

```

❶ Make cache optional

The design above is even better than the previous one because now cache is optional and any other function can call `apply_func` without being forced to do caching by itself. And it works in Numba too!

Reusing cache

Similar to raw outputs, we can force `IndicatorBase.run_pipeline` and `custom_func` to return the cache, so it can be used in other calculations or even indicators. The clear advantage of this approach is that we don't rely on some fixed set of parameter combinations anymore, but on the values of each parameter, which gives us more granularity in managing performance.

```

>>> cache = RelMADist.run(
...     ts2,
...     w1=2,
...     w2=np.arange(2, 1000),
...     return_cache=True)

>>> %timeit RelMADist.run( \
...     ts2, \
...     w1=np.arange(2, 1000), \
...     w2=np.arange(2, 1000), \
...     use_cache=cache)
7.7 ms ± 153 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)

```

Stacking

Similar to regular functions, indicators can depend upon each other. To build a stacked indicator, the first step is merging their inputs and parameters. Consider the old but gold moving average crossover, where we want to use the TA-Lib `SMA` indicator twice: once for the fast and once for the slow moving average. By looking at the arguments accepted by the indicator's `run` method, we see that it accepts a time series `close` and a parameter `timeperiod`. Since both moving averages are computed from the same time series, our only input is `close`. The parameter `timeperiod` should be different for both moving averages, thus we need to define two parameters: `timeperiod1` and `timeperiod2` (you can choose any other names).

```

>>> vbt.phelp(vbt.talib('SMA').run)
SMA.run(
    close,
    timeperiod=Default(value=30),
    short_name='sma',
    hide_params=None,
    hide_default=True,
    **kwargs
):
    Run `SMA` indicator.

    * Inputs: `close`
    * Parameters: `timeperiod`
    * Outputs: `real`

```

Pass a list of parameter names as `'hide_params'` to hide their column levels.
 Set `'hide_default'` to False to show the column levels of the parameters with a default value.
 Other keyword arguments are passed to `'SMA.run_pipeline'`.

```
>>> def apply_func(close, timeperiod1, timeperiod2):
...     fast_ma = vbt.talib('SMA').run(close, timeperiod1)
...     slow_ma = vbt.talib('SMA').run(close, timeperiod2)
...     entries = fast_ma.real_crossed_above(slow_ma)
...     exits = fast_ma.real_crossed_below(slow_ma)
...     return (fast_ma.real, slow_ma.real, entries, exits)

>>> MACrossover = vbt.IF(
...     class_name="CrossSig",
...     input_names=['close'],
...     param_names=['timeperiod1', 'timeperiod2'],
...     output_names=['fast_ma', 'slow_ma', 'entries', 'exits'],
... ).with_apply_func(apply_func)

>>> MACrossover.run(ts2, 2, 3).entries
crosssig_timeperiod1      2
crosssig_timeperiod2      3
                        a  b
2020-01-01             False False
2020-01-02             False False
2020-01-03             False False
2020-01-04             False False
2020-01-05             False  True
2020-01-06             False False
2020-01-07             True  False
```

This implementation has one drawback though: we needlessly create two indicator instances and convert between NumPy arrays and Pandas objects back and forth. An ideal implementation would only use NumPy and Numba. Gladly for us, any indicator constructed by [IndicatorFactory](#) implements the `return_raw` argument, which can be used to access the actual NumPy array(s) returned by the particular calculation function.

```
>>> def sma(close, timeperiod):
...     return vbt.talib('SMA').run(close, timeperiod, return_raw=True)[0][0]

>>> def apply_func(close, timeperiod1, timeperiod2):
...     fast_ma = sma(close, timeperiod1)
...     slow_ma = sma(close, timeperiod2)
...     entries = vbt.nb.crossed_above_nb(fast_ma, slow_ma)
...     exits = vbt.nb.crossed_below_nb(slow_ma, fast_ma)
...     return (fast_ma, slow_ma, entries, exits)
```

Want another approach? Any indicator class created by [IndicatorFactory.with_custom_func](#) has an attribute `custom_func` to access the custom function. Similarly, any indicator class created by [IndicatorFactory.with_apply_func](#) has an attribute `apply_func` to access the apply function. This means that we can call the indicator's `custom_func` from our `custom_func` and the indicator's `apply_func` from our `apply_func`. Just note that `apply_func` of all parsed indicators was created dynamically with `pass_packed` set to True, and thus it accepts arguments in the packed form:

```
>>> vbt.phelp(vbt.talib('SMA').apply_func)
apply_func(
    input_tuple,
    in_output_tuple,
    param_tuple,
    **kwargs
)

>>> def sma(close, timeperiod):
...     return vbt.talib('SMA').apply_func((close,), (), (timeperiod,))
```

That's the fastest it can get!

[Python code](#)

Documentation Indicators

Analysis

To analyze an indicator, we can use the indicator instance returned by the `run` method.

Helper methods

Whenever we create an instance of **IndicatorFactory**, it builds and sets up an indicator class. During this setup, the factory attaches many interesting attributes to the class. For instance, for each in `input_names`, `in_output_names`, `output_names`, and `lazy_outputs`, it will create and attach a bunch of comparison and combination methods. What characteristics any of these attributes should have can be regulated in the `attr_settings` dictionary.

Let's modify the class `CrossSig` created earlier by replacing the entries and exits with a single signal array and also returning an enumerated array that contains the signal type. We will also specify the data type of each array in the `attr_settings` dictionary:

```
>>> from vectorbtpro import *

>>> SignalType = namedtuple('SigType', ['Entry', 'Exit'])(0, 1) ❶

>>> def apply_func(ts, fastw, sloww, minp=None):
...     fast_ma = vbt.nb.rolling_mean_nb(ts, fastw, minp=minp)
...     slow_ma = vbt.nb.rolling_mean_nb(ts, sloww, minp=minp)
...     entries = vbt.nb.crossed_above_nb(fast_ma, slow_ma)
...     exits = vbt.nb.crossed_above_nb(slow_ma, fast_ma)
...     signals = entries | exits
...     signal_type = np.full(ts.shape, -1, dtype=int_) ❷
...     signal_type[entries] = SignalType.Entry
...     signal_type[exits] = SignalType.Exit
...     return (fast_ma, slow_ma, signals, signal_type)

>>> CrossSig = vbt.IF(
...     class_name="CrossSig",
...     input_names=['ts'],
...     param_names=['fastw', 'sloww'],
...     output_names=['fast_ma', 'slow_ma', 'signals', 'signal_type'],
...     attr_settings=dict(
...         fast_ma=dict(dtype=float_),
...         slow_ma=dict(dtype=float_),
...         signals=dict(dtype=np.bool_),
...         signal_type=dict(dtype=SignalType),
...     )
... ).with_apply_func(apply_func)

>>> def generate_index(n):
...     return vbt.date_range("2020-01-01", periods=n)

>>> ts = pd.DataFrame({
...     'a': [1, 2, 3, 2, 1, 2, 3],
...     'b': [3, 2, 1, 2, 3, 2, 1]
... }, index=generate_index(7))
>>> cross_sig = CrossSig.run(ts, 2, 3)
```

❶ Values of an enum in vectorbt usually start with 0 and increase incrementally

❷ A missing value in vectorbt is usually represented with `-1`

We can explore the helper methods that were attached using the Python's `dir` command:

```
>>> dir(cross_sig)
...
'fast_ma',
'fast_ma_above',
'fast_ma_below',
'fast_ma_crossed_above',
'fast_ma_crossed_below',
'fast_ma_equal',
'fast_ma_stats',
...
'signal_type',
'signal_type_readable',
'signal_type_stats',
...
'signals',
'signals_and',
'signals_or',
'signals_stats',
'signals_xor',
...
'slow_ma',
'slow_ma_above',
'slow_ma_below',
'slow_ma_crossed_above',
'slow_ma_crossed_below',
'slow_ma_equal',
'slow_ma_stats',
...
'ts',
```

```
'ts_above',
'ts_below',
'ts_crossed_above',
'ts_crossed_below',
'ts_equal',
'ts_stats',
```

One helper method that appears for each array is `stats`, which calls `StatsBuilderMixin.stats` on the accessor that corresponds to the data type of the array:

```
>>> cross_sig.fast_ma_stats(column=(2, 3, 'a')) ❶
Start      2020-01-01 00:00:00
End        2020-01-07 00:00:00
Period     7 days 00:00:00
Count      6
Mean       2.0
Std        0.547723
Min        1.5
Median     2.0
Max        2.5
Min Index  2020-01-02 00:00:00
Max Index  2020-01-03 00:00:00
Name: (2, 3, a), dtype: object
```

❶ Using `GenericAccessor.stats`

The same can be done manually:

```
>>> cross_sig.fast_ma.vbt.stats(column=(2, 3, 'a'))
```

Numeric

The factory generated the comparison methods `above`, `below`, and `equal` for the numeric arrays. Each of those methods is based on `combine_objs`, which in turn is based on `BaseAccessor.combine`. All operations are done strictly using NumPy. Another advantage is utilization of vectorbt's own broadcasting, such that we can combine the arrays with an arbitrary array-like object, given their shapes can broadcast together. We can also do comparison with multiple objects at once by passing them as a tuple or list.

Let's return True when the fast moving average is above a range of thresholds:

```
>>> cross_sig.fast_ma_above([2, 3])
crosssig_fast_ma_above      2      3
crosssig_fastw              2      2
crosssig_sloww              3      3
      a      b      a      b
2020-01-01      False False False False
2020-01-02      False  True False False
2020-01-03      True  False False False
2020-01-04      True  False False False
2020-01-05      False  True False False
2020-01-06      False  True False False
2020-01-07      True  False False False
```

Or, manually:

```
>>> cross_sig.fast_ma.vbt > vbt.Param([2, 3], name='crosssig_fast_ma_above')
```

Additionally, the factory attached the methods `crossed_above` and `crossed_below`, which are based on `GenericAccessor.crossed_above` and `GenericAccessor.crossed_below` respectively.

```
>>> cross_sig.fast_ma_crossed_above(cross_sig.slow_ma)
crosssig_fastw      2
crosssig_sloww      3
      a      b
2020-01-01      False False
2020-01-02      False False
2020-01-03      False False
2020-01-04      False False
2020-01-05      False  True
2020-01-06      False False
2020-01-07      True  False
```

Or, manually:

```
>>> cross_sig.fast_ma.vbt.crossed_above(cross_sig.slow_ma)
```

Boolean

The factory generated the comparison methods `and`, `or`, and `xor` for the boolean arrays. Similarly to the methods generated for the numeric arrays, they are also based on `combine_objs`:

```
>>> other_signals = pd.Series([False, False, False, False, True, False, False])
>>> cross_sig.signals_and(other_signals)
crosssig_fastw      2
crosssig_sloww      3
      a      b
```

```

2020-01-01    False  False
2020-01-02    False  False
2020-01-03    False  False
2020-01-04    False  False
2020-01-05     True   True
2020-01-06    False  False
2020-01-07    False  False

```

Or, manually:

```
>>> cross_sig.signals.vbt & other_signals
```

Enumerated

Enumerated (or categorical) arrays, such as our `signal_type`, contain integer data that can be mapped to a certain category using a named tuple or any other enum. In contrast to the numeric and boolean arrays, comparing them with other arrays would make no sense. Thus, there is only one attached method - `readable` - that allows us to print the array in a human-readable format:

```

>>> cross_sig.signal_type.readable
crosssig_fastw      2
crosssig_sloww      3
                   a  b
2020-01-01         None None
2020-01-02         None None
2020-01-03         None None
2020-01-04         None None
2020-01-05         Exit Entry
2020-01-06         None None
2020-01-07         Entry Exit

```

Hint

In vectorbt, if `-1` is not listed in the enum, it automatically means a missing value and gets replaced by `None`.

Or, manually:

```
>>> cross_sig.signal_type.vbt(mapping=SignalType).apply_mapping()
```

Indexing

Each indicator class subclasses `Analyzable`, so we can perform Pandas indexing on the indicator instance to select rows and columns in all Pandas objects. Supported operations are `iloc`, `loc`, `xs`, and `__getitem__`.

```

>>> cross_sig = CrossSig.run(ts, [2, 3], [3, 4], param_product=True)

>>> cross_sig.loc["2020-01-03":, (2, 3, 'a')] ❶
<vectorbtpro.indicators.factory.CrossSig at 0x7fc8e2f5c0>

>>> cross_sig.loc["2020-01-03":, (2, 3, 'a')].signals
2020-01-03    False
2020-01-04    False
2020-01-05     True
2020-01-06    False
2020-01-07     True
Name: (2, 3, a), dtype: bool

```

❶ Using one Pandas indexing operation

Additionally, `IndicatorFactory` uses the class factory function `build_param_indexer` to create an indexing class that enables Pandas indexing on each defined parameter. Since the indicator class subclasses this indexing class, we can use `*param_name*_loc` to select one or more values of any parameter.

```

>>> cross_sig.fastw_loc[2].sloww_loc[3]['a'] ❶
<vectorbtpro.indicators.factory.CrossSig at 0x7fcac80742b0>

>>> cross_sig.fastw_loc[2].sloww_loc[3]['a'].signals
2020-01-01    False
2020-01-02    False
2020-01-03    False
2020-01-04    False
2020-01-05     True
2020-01-06    False
2020-01-07     True
Name: a, dtype: bool

```

❶ Using two parameter and one Pandas indexing operation. Each of those operations returns a separate instance of `CrossSig`.

This all makes possible accessing rows and columns by labels, integer positions, and parameters - full flexibility 🧙

Stats and plots

As with every `Analyzable` instance, we can compute and plot various properties of the (input and output) data stored in the instance.

Metrics can be defined in two ways: by passing them via the `metrics` argument, or by subclassing the indicator class. The same applies to the `stats_defaults` argument, which can be provided either as a dictionary or a function, and which defines the default settings for `StatsBuilderMixin.stats`. Subplots can be defined in a similar way to metrics, except that they are set up by `subplots` and `plots_defaults` arguments, and invoked by `PlotsBuilderMixin.plots`.

Let's define some metrics and subplots for `CrossSig`:

```
>>> metrics = dict(
...     start=dict( ❶
...         title='Start',
...         calc_func=lambda self: self.wrapper.index[0],
...         agg_func=None
...     ),
...     end=dict( ❷
...         title='End',
...         calc_func=lambda self: self.wrapper.index[-1],
...         agg_func=None
...     ),
...     period=dict( ❸
...         title='Period',
...         calc_func=lambda self: len(self.wrapper.index),
...         apply_to_timedelta=True,
...         agg_func=None
...     ),
...     fast_stats=dict( ❹
...         title="Fast Stats",
...         calc_func=lambda self:
...             self.fast_ma.describe()\
...             .loc[['count', 'mean', 'std', 'min', 'max']]\
...             .vbt.to_dict(orient='index_series')
...     ),
...     slow_stats=dict(
...         title="Slow Stats",
...         calc_func=lambda self:
...             self.slow_ma.describe()\
...             .loc[['count', 'mean', 'std', 'min', 'max']]\
...             .vbt.to_dict(orient='index_series')
...     ),
...     num_entries=dict( ❺
...         title="Entries",
...         calc_func=lambda self:
...             np.sum(self.signal_type == SignalType.Entry)
...     ),
...     num_exits=dict(
...         title="Exits",
...         calc_func=lambda self:
...             np.sum(self.signal_type == SignalType.Exit)
...     )
... )

>>> def plot_mas(self, column=None, add_trace_kwargs=None, fig=None): ❻
...     ts = self.select_col_from_obj(self.ts, column).rename('TS') ❼
...     fast_ma = self.select_col_from_obj(self.fast_ma, column).rename('Fast MA')
...     slow_ma = self.select_col_from_obj(self.slow_ma, column).rename('Slow MA')
...     ts.vbt.plot(add_trace_kwargs=add_trace_kwargs, fig=fig)
...     fast_ma.vbt.plot(add_trace_kwargs=add_trace_kwargs, fig=fig) ❽
...     slow_ma.vbt.plot(add_trace_kwargs=add_trace_kwargs, fig=fig)

>>> def plot_signals(self, column=None, add_trace_kwargs=None, fig=None):
...     signal_type = self.select_col_from_obj(self.signal_type, column)
...     entries = (signal_type == SignalType.Entry).rename('Entries')
...     exits = (signal_type == SignalType.Exit).rename('Exits')
...     entries.vbt.plot(add_trace_kwargs=add_trace_kwargs, fig=fig)
...     exits.vbt.plot(add_trace_kwargs=add_trace_kwargs, fig=fig)

>>> subplots = dict(
...     mas=dict(
...         title="Moving averages",
...         plot_func=plot_mas
...     ),
...     signals=dict(
...         title="Signals",
...         plot_func=plot_signals
...     )
... )

>>> CrossSig = vbt.IF(
...     class_name="CrossSig",
...     input_names=['ts'],
...     param_names=['fastw', 'sloww'],
...     output_names=['fast_ma', 'slow_ma', 'signals', 'signal_type'],
...     attr_settings=dict(
...         fast_ma=dict(dtype=float_),
...         slow_ma=dict(dtype=float_),
...         signals=dict(dtype=np.bool_),
...         signal_type=dict(dtype=SignalType),
...     ),
...     metrics=metrics, ❾
...     subplots=subplots
... ).with_apply_func(apply_func)

>>> cross_sig = CrossSig.run(ts, [2, 3], 4)
```

❶ Index of the first data point

❷ Index of the last data point

- 3 The number of data points multiplied by the index frequency
- 4 Takes `fast_ma`, generates the descriptive statistics using Pandas, and converts them to a dictionary with Series (of columns) per statistic using `BaseAccessor.to_dict`. The `dict` type instructs vectorbt to make multiple metrics out of one.
- 5 The number of entry signals
- 6 `PlotsBuilderMixin.plots` can automatically recognize the arguments the plotting function takes, and pass the necessary information. For example, by seeing `self`, it will pass the indicator instance.
- 7 Since we are already plotting multiple time series, we cannot plot multiple columns at once, so we must select the provided column from each time series using `Wrapping.select_col_from_obj`
- 8 `fig` contains the figure and `add_trace_kwargs` contains the specification for the subplot this time series should be plotted over. Note how the function returns nothing: the figure is being modified in-place.
- 9 Metric and subplot dictionaries are passed to the constructor

Calculate the metrics:

```
>>> cross_sig.stats(column=(2, 4, 'a'))
Start      2020-01-01 00:00:00
End        2020-01-07 00:00:00
Period      7 days 00:00:00
Fast Stats: count      6.0
Fast Stats: mean       2.0
Fast Stats: std        0.547723
Fast Stats: min        1.5
Fast Stats: max        2.5
Slow Stats: count      4.0
Slow Stats: mean       2.0
Slow Stats: std        0.0
Slow Stats: min        2.0
Slow Stats: max        2.0
Entries      1
Exits        1
Name: (2, 4, a), dtype: object
```

Plot the subplots:

```
>>> cross_sig.plots(column=(2, 4, 'a')).show()
```



We have created a smart indicator, yay! 🎉

Extending

Indicator classes can be extended and modified just as regular Python classes - by subclassing. Let's make the newly created functions `plot_mas` and `plot_signals` methods of the indicator class, such that we can plot both graphs separately. We will also redefine the `subplots` config to account for this change:

```
>>> class SmartCrossSig(CrossSig):
...     def plot_mas(self, column=None, add_trace_kwargs=None, fig=None):
...         ts = self.select_col_from_obj(self.ts, column).rename('TS')
...         fast_ma = self.select_col_from_obj(self.fast_ma, column).rename('Fast MA')
...         slow_ma = self.select_col_from_obj(self.slow_ma, column).rename('Slow MA')
...         fig = ts.vbt.plot(add_trace_kwargs=add_trace_kwargs, fig=fig)
```

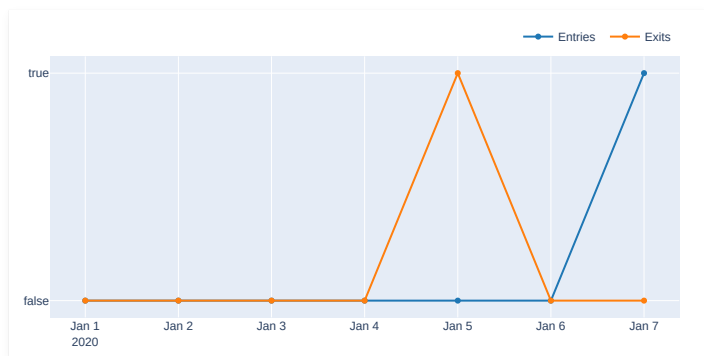
```

...     fast_ma.vbt.plot(add_trace_kwargs=add_trace_kwargs, fig=fig)
...     slow_ma.vbt.plot(add_trace_kwargs=add_trace_kwargs, fig=fig)
...     return fig ❶
...
...     def plot_signals(self, column=None, add_trace_kwargs=None, fig=None):
...         signal_type = self.select_col_from_obj(self.signal_type, column)
...         entries = (signal_type == SignalType.Entry).rename('Entries')
...         exits = (signal_type == SignalType.Exit).rename('Exits')
...         fig = entries.vbt.plot(add_trace_kwargs=add_trace_kwargs, fig=fig)
...         exits.vbt.plot(add_trace_kwargs=add_trace_kwargs, fig=fig)
...         return fig
...
...     subplots = vbt.HybridConfig( ❷
...         mas=dict(
...             title="Moving averages",
...             plot_func='plot_mas' ❸
...         ),
...         signals=dict(
...             title="Signals",
...             plot_func='plot_signals'
...         )
...     )

>>> smart_cross_sig = SmartCrossSig.run(ts, [2, 3], 4)
>>> smart_cross_sig.plot_signals(column=(2, 4, 'a')).show()

```

- ❶ Since both plotting functions can now be invoked by the user, we need to return the figure in case it wasn't provided
- ❷ We need to use `Config` instead of `dict` when setting `metrics` and `subplots` manually
- ❸ Since both functions have become attributes of the class, we can specify them as strings to make use of vectorbt's [attribute resolution](#)



[Python code](#)

[Documentation](#)
[Indicators](#)

<> Parsers

IndicatorFactory deploys a collection of parsers to simplify creation of indicators, ranging from third-party indicator parsers to a powerful expression parser.

Info

Each parser method is a class method with the prefix `from_`, meaning we don't have to construct and pass any information to the indicator factory using `vbt.IF(...)` - the method already does it for us!

TA-Lib

IndicatorFactory.from_talib can parse **TA-Lib** indicators. Whenever we pass the name of an indicator, the method gets the TA-Lib abstract function, and then looks in the `info` dictionary to derive the input, parameter, and output names. After constructing a factory instance, it builds an apply function that can run the indicator function on two-dimensional inputs (as opposed to one-dimensional inputs that are currently supported by TA-Lib).

To get the list of all supported indicators:

```
>>> from vectorbtpro import *

>>> vbt.IF.list_talib_indicators()
{'ACOS',
 'AD',
 'ADD',
 ...,
 'WCLPRICE',
 'WILLR',
 'WMA'}
```

1 Using IndicatorFactory.list_talib_indicators

To get an indicator:

```
>>> vbt.IF.from_talib('RSI')
vectorbtpro.indicators.factory.talib.RSI
```

1 Using IndicatorFactory.from_talib

Or, using a shortcut:

```
>>> vbt.talib('RSI')
vectorbtpro.indicators.factory.talib.RSI
```

Skipping NaN

TA-Lib indicators are usually very unhappy when they encounter missing data. For instance, a single NaN in a time series can propagate this NaN to all data points that follow:

```
>>> price = vbt.RandomData.pull(
...     start='2020-01-01',
...     end='2020-06-01',
...     timeframe='1H',
...     seed=42
... ).get()
>>> price_na = price.copy()
>>> price_na.iloc[2] = np.nan

>>> SMA = vbt.talib("SMA")
>>> sma = SMA.run(price_na, timeperiod=10)
>>> sma.real
2019-12-31 22:00:00+00:00    NaN
2019-12-31 23:00:00+00:00    NaN
2020-01-01 00:00:00+00:00    NaN
2020-01-01 01:00:00+00:00    NaN
2020-01-01 02:00:00+00:00    NaN
...
2020-05-31 18:00:00+00:00    NaN
2020-05-31 19:00:00+00:00    NaN
2020-05-31 20:00:00+00:00    NaN
2020-05-31 21:00:00+00:00    NaN
2020-05-31 22:00:00+00:00    NaN
Freq: H, Name: 10, Length: 3649, dtype: float64
```

1 Make a single value missing

To address this, we can tell the indicator factory to run the indicator on non-NA values only and then place the output values at their original positions:

```
>>> sma = SMA.run(price_na, timeperiod=10, skipna=True)
>>> sma.real
```

```

2019-12-31 22:00:00+00:00      NaN
2019-12-31 23:00:00+00:00      NaN
2020-01-01 00:00:00+00:00      NaN
2020-01-01 01:00:00+00:00      NaN
2020-01-01 02:00:00+00:00      NaN
...
2020-05-31 18:00:00+00:00    213.169260
2020-05-31 19:00:00+00:00    212.477181
2020-05-31 20:00:00+00:00    211.911416
2020-05-31 21:00:00+00:00    211.310849
2020-05-31 22:00:00+00:00    210.899923
Freq: H, Name: 10, Length: 3649, dtype: float64

```

Hint

Another option would be to forward fill NaN values before running an indicator, but this would skew the results, thus make use of this option whenever this is really appropriate.

Resampling

Another feature implemented by the indicator factory is the support for parametrized time frames!

This works the following way:

1. Using the wrapper, input arrays are downsampled to the target time frame
2. Indicator is run on downsampled input arrays
3. Output arrays are upsampled back to the original time frame

This way, multiple time frames can be packed into a single two-dimensional array:

```

>>> sma = SMA.run(
...     price_na,
...     timeperiod=10,
...     skipna=True,
...     timeframe=["1h", "4h", "1d"]
... )
>>> sma.real
sma_timeperiod      10
sma_timeframe      1h      4h      1d
2019-12-31 22:00:00+00:00      NaN      NaN      NaN
2019-12-31 23:00:00+00:00      NaN      NaN      NaN
2020-01-01 00:00:00+00:00      NaN      NaN      NaN
2020-01-01 01:00:00+00:00      NaN      NaN      NaN
2020-01-01 02:00:00+00:00      NaN      NaN      NaN
...
2020-05-31 18:00:00+00:00    213.169260    215.561805    206.104351
2020-05-31 19:00:00+00:00    212.477181    214.422456    206.104351
2020-05-31 20:00:00+00:00    211.911416    214.422456    206.104351
2020-05-31 21:00:00+00:00    211.310849    214.422456    206.104351
2020-05-31 22:00:00+00:00    210.899923    214.422456    206.104351

[3649 rows x 3 columns]

```

Note

When some timestamps are missing, vectorbt may have difficulties parsing the frequency of the source index. To provide the frequency explicitly, pass `broadcast_kwargs=dict(wrapper_kwargs=dict(freq="1h"))`, for example. Without the source frequency, vectorbt will upsample the downsampled arrays between each two timestamps in the source index instead of relying on its frequency, which may be undesired.

Plotting

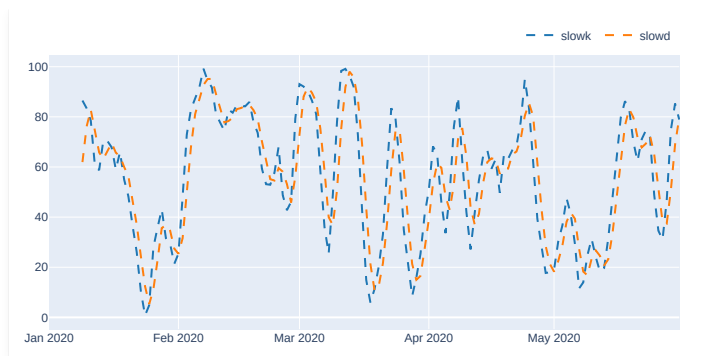
Additionally, we can plot each indicator. This is achieved fully programmatically by parsing the indicator's output flags. Let's take `STOCH` as an example:

```

>>> STOCH = vbt.talib('STOCH')
>>> STOCH.output_flags
OrderedDict([('slowk', ['Dashed Line']), ('slowd', ['Dashed Line'])])

>>> ohlc = price.resample('1d').ohlc()
>>> stoch = STOCH.run(ohlc['high'], ohlc['low'], ohlc['close'])
>>> stoch.plot().show()

```

To see what arguments the `plot` function takes, use [phelp](#):

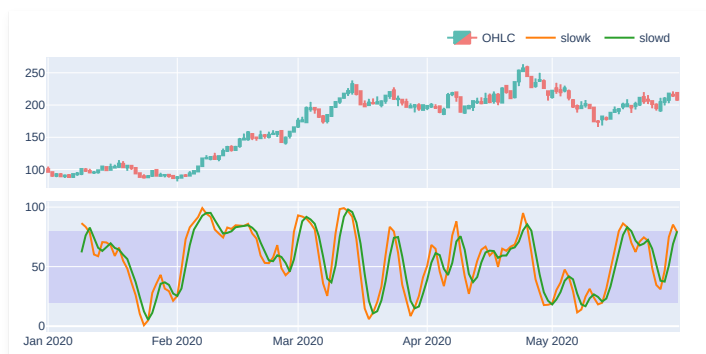
```
>>> vbt.phelp(STOCH.plot)
plot(
    self,
    column=None,
    limits=None,
    add_shape_kwargs=None,
    add_trace_kwargs=None,
    fig=None,
    slowk_trace_kwargs=None,
    slowd_trace_kwargs=None,
    **layout_kwargs
):
    Plot the outputs of the indicator based on their flags.

    Args:
        column (str): Name of the column to plot.
        limits (tuple of float): Tuple of the lower and upper limit.
        add_shape_kwargs (dict): Keyword arguments passed to 'fig.add_shape' when adding the range between both limits.
        add_trace_kwargs (dict): Keyword arguments passed to 'fig.add_trace' when adding each trace.
        slowk_trace_kwargs (dict): Keyword arguments passed to the trace of 'slowk'.
        slowd_trace_kwargs (dict): Keyword arguments passed to the trace of 'slowd'.
        fig (Figure or FigureWidget): Figure to add the traces to.
        **layout_kwargs: Keyword arguments passed to 'fig.update_layout'.
```

Let's create a plot with two subplots: OHLC above, and %D and %K below. We will also change the style of both output lines from dashed to solid, and display a range between an oversold limit of 20 and an overbought limit of 80:

```
>>> fig = vbt.make_subplots(
...     rows=2,
...     cols=1,
...     shared_xaxes=True, ①
...     vertical_spacing=0.05)
>>> ohlc.vbt.ohlc.plot(
...     add_trace_kwargs=dict(row=1, col=1), ②
...     fig=fig,
...     xaxis=dict(rangeslider_visible=False)) ③
>>> stoch.plot(
...     limits=(20, 80),
...     add_trace_kwargs=dict(row=2, col=1), ④
...     slowk_trace_kwargs=dict(line=dict(dash=None)), ⑤
...     slowd_trace_kwargs=dict(line=dict(dash=None)),
...     fig=fig)
>>> fig.show()
```

- ① All time series have the same index
- ② First subplot
- ③ Passed directly to `fig.update_layout`
- ④ Second subplot
- ⑤ Passed directly to `fig.add_trace`



Pandas TA

IndicatorFactory.from_pandas_ta can parse **Pandas TA** indicators. Since Pandas TA indicators have not metadata attached to each indicator, there is a method **IndicatorFactory.parse_pandas_ta_config** that reads the signature of an indicator function to get the input and parameter names and defaults, passes several dozens of rows of sample data to the function, and derives the number and names of the outputs.

Note

If any indicator raises an error while parsing, try increasing the number of rows passed to the indicator function, for example, by passing `parse_kwargs=dict(test_index_len=150)`.

To get the list of all supported indicators:

```
>>> vbt.IF.list_pandas_ta_indicators()
{'ABERRATION',
 'ACCBANDS',
 'AD',
 ...
 'X SIGNALS',
 'ZLMA',
 'ZSCORE'}
```

1 Using **IndicatorFactory.list_pandas_ta_indicators**

To get an indicator:

```
>>> vbt.IF.from_pandas_ta('RSI')
vectorbtpro.indicators.factory.pandas_ta.RSI
```

1 Using **IndicatorFactory.from_pandas_ta**

Or, using a shortcut:

```
>>> vbt.pandas_ta('RSI')
vectorbtpro.indicators.factory.pandas_ta.RSI
```

TA

IndicatorFactory.from_ta can parse **TA** indicators. Similarly to Pandas TA, TA indicators must be explicitly parsed to get the context of each indicator function. Since every indicator is a class, there is a method **IndicatorFactory.parse_ta_config** that reads the signature, the docstring, and the attributes of the class to derive the input, parameter, and output names and defaults.

To get the list of all supported indicators:

```
>>> vbt.IF.list_ta_indicators()
{'ADXIndicator',
 'AccDistIndexIndicator',
 'AroonIndicator',
 ...
 'VortexIndicator',
 'WMAIndicator',
 'WilliamsRIndicator'}
```

1 Using **IndicatorFactory.list_ta_indicators**

To get an indicator:

```
>>> vbt.IF.from_ta('RSIIndicator')
vectorbtpro.indicators.factory.ta.RSIIndicator
```

1 Using **IndicatorFactory.from_ta**

Or, using a shortcut:

```
>>> vbt.ta('RSIIndicator')
vectorbtpro.indicators.factory.ta.RSIIndicator
```

Expressions

Expressions are a brand-new way to define indicators of any complexity using regular strings. The main advantage of expressions over custom and apply functions is that vectorbt can easily introspect the code of an indicator and inject a lot of useful automations.

Expressions are converted into full-blown indicators by a hybrid method **IndicatorFactory.from_expr**. Why hybrid? It's both a class and an instance method. We can call this method on an instance in case when we want to have a full control over the indicator's specification, and on a class in case when we want the entire specification to be parsed for us. Let's try both approaches while building an ATR indicator!

Instance method

Here's a semi-automated implementation using the instance method:

```

>>> expr = """
... tr0 = abs(high - low)
... tr1 = abs(high - fshift(close))
... tr2 = abs(low - fshift(close))
... tr = nanmax(column_stack((tr0, tr1, tr2)), axis=1)
... atr = wmm_mean_1d(tr, n)
... tr, atr
... """
>>> ATR = vbt.IF(
...     class_name='ATR',
...     input_names=['high', 'low', 'close'],
...     param_names=['n'],
...     output_names=['tr', 'atr']
... ).from_expr(expr, n=14)

>>> atr = ATR.run(ohlcv['high'], ohlcv['low'], ohlcv['close'])
>>> atr.atr
2019-12-31 00:00:00+00:00      NaN
2020-01-01 00:00:00+00:00      NaN
2020-01-02 00:00:00+00:00      NaN
2020-01-03 00:00:00+00:00      NaN
2020-01-04 00:00:00+00:00      NaN
...
2020-05-27 00:00:00+00:00    13.394434
2020-05-28 00:00:00+00:00    13.338482
2020-05-29 00:00:00+00:00    13.480809
2020-05-30 00:00:00+00:00    13.003231
2020-05-31 00:00:00+00:00    12.888624
Freq: D, Length: 153, dtype: float64

```

The expression `expr` is just a regular Python code without any extensions that gets evaluated using the Python's `eval` command. All function names are resolved by the parser prior to the evaluation.

Class method

And here's a fully-automated implementation using the class method and annotations:

```

>>> expr = """
... ATR:
... tr0 = abs(@in_high - @in_low)
... tr1 = abs(@in_high - fshift(@in_close))
... tr2 = abs(@in_low - fshift(@in_close))
... @out_tr = nanmax(column_stack((tr0, tr1, tr2)), axis=1)
... @out_atr = wmm_mean_1d(@out_tr, @p_n)
... @out_tr, @out_atr
... """
>>> ATR = vbt.IF.from_expr(expr, n=14)

```

In the first example, we provided all the required information manually by constructing a new instance of [IndicatorFactory](#). The parser identified each of the input and parameter names in the expression, and replaced them by the actual arrays. In the second example, we used annotations to give to the parser hints about the meaning of each variable. Whenever the parser finds a substring starting with `@`, it knows that it has a special meaning for constructing a factory instance. For instance, the prefixes `@_in`, `@_p`, and `@_out` indicate an input, parameter, and output respectively. The names appear in the order they appear in the expression (apart from OHLCV, where H comes always after O):

```

>>> ATR.input_names
('high', 'low', 'close')

```

But the parser can even parse information that doesn't start with a special character. For example, each of `open`, `high`, `low`, `close`, and `volume` are identified automatically, such that we don't need to provide annotations for them. They are referred to as magnet inputs, which are specified via the `magnet_inputs` argument. In case none of the outputs have annotations and the expression is a multi-line expression with the last line containing a tuple with valid variable names, we don't even have to provide annotations for the outputs. Also, as we saw above, the class name can be provided in the first line trailed by a colon:

```

>>> expr = """
... ATR:
... tr0 = abs(high - low)
... tr1 = abs(high - fshift(close))
... tr2 = abs(low - fshift(close))
... tr = nanmax(column_stack((tr0, tr1, tr2)), axis=1)
... atr = wmm_mean_1d(tr, @p_n)
... tr, atr
... """
>>> ATR = vbt.IF.from_expr(expr, n=14)
>>> ATR.input_names
('high', 'low', 'close')

>>> ATR.output_names
('tr', 'atr')

```

What about functions? The parser can identify functions by looking into various modules and packages. In our example, `abs` and `nanmax` have been found in NumPy, while `wmm_mean_1d` has been found among generic Numba-compiled functions in `nb` (even without the `_nb` suffix). Look in the API of [IndicatorFactory.from_expr](#) to learn more. To avoid naming conflicts, we can still access the NumPy, Pandas, and vectorbt modules via `np`, `pd`, and `vbt` respectively:

```

>>> expr = """
... ATR:
... tr0 = abs(high - low)
... tr1 = abs(high - vbt.nb.fshift_nb(close))

```

```
... tr2 = abs(low - vbt.nb.fshift_nb(close))
... tr = np.nanmax(np.column_stack((tr0, tr1, tr2)), axis=1)
... atr = vbt.nb.wmm_mean_1d_nb(tr, n)
... tr, atr
... """
```

TA-Lib

Another automation touches TA-Lib indicators: vectorbt will replace any variable annotated with `@talib` with an actual TA-Lib indicator function that can work on both one-dimensional and two-dimensional data!

```
>>> expr = """
... ATR:
... tr0 = abs(high - low)
... tr1 = abs(high - fshift(close))
... tr2 = abs(low - fshift(close))
... tr = nanmax(column_stack((tr0, tr1, tr2)), axis=1)
... atr = @talib_ema(tr, 2 * n - 1) # Wilder's EMA
... tr, atr
... """
```

Context

So, how can we define our own functions and rules? Any additional keyword argument passed to `IndicatorFactory.from_expr` acts as a context for the evaluation and can replace a variable with the same name. Let's define our own function `shift_close` that acts as an alias for `fshift_nb`:

```
>>> expr = """
... ATR:
... tr0 = abs(high - low)
... tr1 = abs(high - shift_close(close))
... tr2 = abs(low - shift_close(close))
... tr = nanmax(column_stack((tr0, tr1, tr2)), axis=1)
... atr = wmm_mean_1d(tr, @p_n)
... tr, atr
... """
>>> ATR = vbt.IF.from_expr(expr, n=14, shift_close=vbt.nb.fshift_nb)
```

We may have functions that depend on the evaluation context. In the example above, we can make our `shift_close` accept the context and pull the number of periods to shift the closing price by (just for the sake of example):

```
>>> def shift_close(close, context):
...     return vbt.nb.fshift_nb(close, context.get('shift', 1))

>>> expr = """
... ATR:
... tr0 = abs(high - low)
... tr1 = abs(high - shift_close(close))
... tr2 = abs(low - shift_close(close))
... tr = nanmax(column_stack((tr0, tr1, tr2)), axis=1)
... atr = wmm_mean_1d(tr, @p_n)
... tr, atr
... """
>>> ATR = vbt.IF.from_expr(expr, n=14, shift_close=shift_close, shift=2)
```

The context will be automatically passed to the function once `context` has been recognized in its arguments. Moreover, we can make our `shift_close` to also pull the closing price itself. Notice how `shift_close` takes no more arguments in the expression:

```
>>> def shift_close(context):
...     return vbt.nb.fshift_nb(context['close'], context.get('shift', 1))

>>> expr = """
... ATR:
... tr0 = abs(high - low)
... tr1 = abs(high - shift_close())
... tr2 = abs(low - shift_close())
... tr = nanmax(column_stack((tr0, tr1, tr2)), axis=1)
... atr = wmm_mean_1d(tr, @p_n)
... tr, atr
... """
>>> ATR = vbt.IF.from_expr(expr, n=14, shift_close=shift_close)
```

If we run this, we'll get an error stating that `close` was not found in the context. This is because the input `close` is not "visible" in the expression, so it wasn't appended to the list of input names. To make any input, in-output, or parameter visible even without including it in the expression, we need to notify vectorbt that there is a function depending on it by using a dictionary called `func_mapping`, which takes functions and the magnet names they depend on:

```
>>> func_mapping = dict(
...     shift_close=dict(
...         func=shift_close,
...         magnet_inputs=['close']
...     )
... )
>>> ATR = vbt.IF.from_expr(expr, n=14, func_mapping=func_mapping)
```

Since `shift_close` depends only on the context, we can instruct the parser to call it before the evaluation and only once, which would effectively cache it. For this, we need to use `res_func_mapping` instead of `func_mapping`:

```

>>> expr = """
... ATR:
... tr0 = abs(high - low)
... tr1 = abs(high - shifted_close)
... tr2 = abs(low - shifted_close)
... tr = nanmax(column_stack((tr0, tr1, tr2)), axis=1)
... atr = wmm_mean_1d(tr, @p_n)
... tr, atr
... """
>>> res_func_mapping = dict(
...     shifted_close=dict(
...         func=shift_close,
...         magnet_inputs=['close']
...     )
... )
>>> ATR = vbt.IF.from_expr(expr, n=14, res_func_mapping=res_func_mapping)

```

Notice how `shifted_close` doesn't have parentheses anymore - it has become an array.

Settings

But that's not all. How about overriding any information passed to `IndicatorFactory.from_expr` but from within an expression? Wonder or not, this is also possible! We can define a dictionary anywhere in the expression annotated with `@settings({...})`. The dictionary within the parentheses will be evaluated with the Python `eval` command prior to the main evaluation and merged over the default settings of the factory method.

Let's rewrite the [instance method](#) example solely using an expression:

```

>>> expr = """
... @settings(dict(
...     factory_kwargs=dict(
...         class_name='ATR',
...         input_names=['high', 'low', 'close'],
...         param_names=['n'],
...         output_names=['tr', 'atr']
...     ),
...     n=14
... ))
...
... tr0 = abs(high - low)
... tr1 = abs(high - fshift(close))
... tr2 = abs(low - fshift(close))
... tr = nanmax(column_stack((tr0, tr1, tr2)), axis=1)
... atr = wmm_mean_1d(tr, n)
... tr, atr
... """
>>> ATR = vbt.IF.from_expr(expr)

```

Stacking

Remember that we can use arbitrary Python code inside our expressions, even other indicators. To simplify the usage of indicators, there is a convenient annotation `@res`, which takes the name of an indicator and creates an automatically resolved function out of it, just like `shifted_close` above. It makes this function an entry of `res_func_mapping` and puts the indicator's input, in-output, and parameter names into the entry's magnet lists, so we don't have to worry about passing the right information to the indicator - vectorbt does it for us!

Let's illustrate this awesomeness by defining basic SuperTrend bands:

```

>>> expr = """
... SuperTrend[st]:
... avg_price = (high + low) / 2
... up = avg_price + @p_mult * @res_talib_atr
... down = avg_price - @p_mult * @res_talib_atr
... up, down
... """
>>> SuperTrend = vbt.IF.from_expr(expr, mult=3, atr_timeperiod=10)

>>> SuperTrend.input_names
('high', 'low', 'close')

>>> SuperTrend.param_names
('mult', 'atr_timeperiod')

>>> SuperTrend.output_names
('up', 'down')

>>> st = SuperTrend.run(ohlcv['high'], ohlc['low'], ohlc['close'])
>>> st.up
2019-12-31 00:00:00+00:00      NaN
2020-01-01 00:00:00+00:00      NaN
2020-01-02 00:00:00+00:00      NaN
2020-01-03 00:00:00+00:00      NaN
2020-01-04 00:00:00+00:00      NaN
...
2020-05-27 00:00:00+00:00    240.698333
2020-05-28 00:00:00+00:00    245.683118
2020-05-29 00:00:00+00:00    251.211675
2020-05-30 00:00:00+00:00    256.406868
2020-05-31 00:00:00+00:00    250.201819
Freq: D, Length: 153, dtype: float64

```

- 1 Short name of the indicator can be provided in the square brackets following the class name
- 2 The TA-Lib's ATR indicator depends on `high`, `low`, and `close`
- 3 Our indicator depends upon the multiplier `mult`, while the ATR also depends upon `timeperiod`.

So, what happens if there are two indicators with overlapping inputs, parameters, or other arguments? Every argument apart from the inputs receives a prefix with the short name of the indicator (`IndicatorBase.short_name`). Under the hood, `vectorbt` traverses the signature of the indicator's `run` method, and looks whether there is an argument with the same name in the context (and don't forget the prefix).

By default, the resolved function returns a raw output in form of one or more NumPy arrays. If the indicator has more than one output, we can use regular indexing to select a specific array, such as `@res_talib_macd[0]`. Let's disable raw outputs for ATR and query the `real` Pandas object on its indicator instance instead:

```
>>> expr = """
... SuperTrend[st]:
... avg_price = (high + low) / 2
... up = avg_price + @p_mult * @res_talib_atr.real.values
... down = avg_price - @p_mult * @res_talib_atr.real.values
... up, down
... """
>>> SuperTrend = vbt.IF.from_expr(
...     expr,
...     mult=3,
...     atr_timeperiod=10,
...     atr_kwargs=dict(return_raw=False)) ❶
```

- 1 If we printed `vbt.pheip(vbt.talib('ATR').run)`, we would see that any additional keyword argument is passed as `**kwargs`, so we can specify `atr_kwargs` to target those variable arguments

One-liners

There is nothing more satisfying than being able to define an indicator in one line 😊

```
>>> AvgPrice = vbt.IF.from_expr("AvgPrice: @out_avg_price:(high + low) / 2")

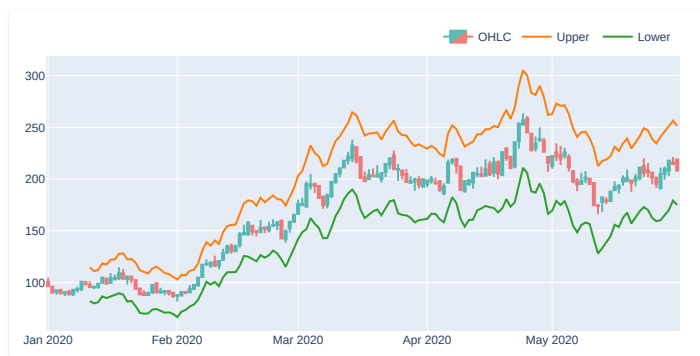
>>> AvgPrice.run(ohlc['high'], ohlc['low']).avg_price
2019-12-31 00:00:00+00:00    100.496714
2020-01-01 00:00:00+00:00    100.216791
2020-01-02 00:00:00+00:00     92.735610
2020-01-03 00:00:00+00:00     90.353978
2020-01-04 00:00:00+00:00     91.676538
...
2020-05-27 00:00:00+00:00    200.426358
2020-05-28 00:00:00+00:00    205.655007
2020-05-29 00:00:00+00:00    210.587055
2020-05-30 00:00:00+00:00    217.806298
2020-05-31 00:00:00+00:00    212.041686
Freq: D, Length: 153, dtype: float64
```

Notice how the output annotation `@out` isn't bound to any variable anymore but is written similarly to the class name - with a trailing colon following the expression of the output. If there are multiple outputs, their output expressions must be separated by a comma. Here's a single-line expression for basic SuperTrend bands with multiple outputs:

```
>>> SuperTrend = vbt.IF.from_expr(
...     "SuperTrend[st]: @out_up:@res_avg_price + @p_mult * @res_talib_atr, "
...     "@out_down:@res_avg_price - @p_mult * @res_talib_atr", ❶
...     avg_price=AvgPrice,
...     atr_timeperiod=10,
...     mult=3)
>>> st = SuperTrend.run(ohlc['high'], ohlc['low'], ohlc['close'])

>>> fig = ohlc.vbt.ohlcv.plot()
>>> st.up.rename('Upper').vbt.plot(fig=fig)
>>> st.down.rename('Lower').vbt.plot(fig=fig)
>>> fig.show()
```

- 1 Split the expression for better readability



Using Pandas

Like many other factory methods, `IndicatorFactory.from_expr` passes inputs and in-outputs as two-dimensional NumPy arrays. We can enable the `keep_pd` flag to work on Pandas objects. Let's run our ATR indicator using Pandas alone:

```
>>> expr = """
... ATR:
... tr0 = abs(high - low)
... tr1 = abs(high - close.shift())
... tr2 = abs(low - close.shift())
... tr = pd.concat((tr0, tr1, tr2), axis=1).max(axis=1)
... atr = tr.ewm(alpha=1 / @p_n, adjust=False, min_periods=@p_n).mean()
... tr, atr
... """
>>> ATR = vbt.IF.from_expr(expr, n=14, keep_pd=True)
>>> atr = ATR.run(ohlc['high'], ohlc['low'], ohlc['close'])
>>> atr.atr
2019-12-31 00:00:00+00:00      NaN
2020-01-01 00:00:00+00:00      NaN
2020-01-02 00:00:00+00:00      NaN
2020-01-03 00:00:00+00:00      NaN
2020-01-04 00:00:00+00:00      NaN
...
2020-05-27 00:00:00+00:00    13.394434
2020-05-28 00:00:00+00:00    13.338482
2020-05-29 00:00:00+00:00    13.480809
2020-05-30 00:00:00+00:00    13.003231
2020-05-31 00:00:00+00:00    12.888624
Freq: D, Length: 153, dtype: float64
```

Note

In contrast to the previous NumPy-only expressions, this expression won't work for multiple columns of input data.

For simpler expressions, we can instruct the parser to use `pandas.eval` instead of the Python's `eval`. This brings multi-threading and other performance advantages for big inputs since `pd.eval` switches to `NumExpr` by default:

```
>>> AvgPrice = vbt.IF.from_expr(
...     "AvgPrice: @out_avg_price:(high + low) / 2",
...     use_pd_eval=True
... )
>>> AvgPrice.run(ohlc['high'], ohlc['low']).avg_price
2019-12-31 00:00:00+00:00    100.496714
2020-01-01 00:00:00+00:00    100.216791
2020-01-02 00:00:00+00:00     92.735610
2020-01-03 00:00:00+00:00     90.353978
2020-01-04 00:00:00+00:00     91.676538
...
2020-05-27 00:00:00+00:00    200.426358
2020-05-28 00:00:00+00:00    205.655007
2020-05-29 00:00:00+00:00    210.587055
2020-05-30 00:00:00+00:00    217.006298
2020-05-31 00:00:00+00:00    212.041686
Freq: D, Length: 153, dtype: float64
```

Debugging

To see the expression after parsing all the annotations, set `return_clean_expr` to `True`:

```
>>> expr = """
... ATR:
... tr0 = abs(@in_high - @in_low)
... tr1 = abs(@in_high - fshift(@in_close))
... tr2 = abs(@in_low - fshift(@in_close))
... @out_tr = nanmax(column_stack((tr0, tr1, tr2)), axis=1)
... @out_atr = wwm_mean_1d(@out_tr, @p_n)
... @out_tr, @out_atr
... """
>>> print(vbt.IF.from_expr(expr, n=14, return_clean_expr=True))
tr0 = abs(__in_high - __in_low)
tr1 = abs(__in_high - fshift(__in_close))
tr2 = abs(__in_low - fshift(__in_close))
__out_tr = nanmax(column_stack((tr0, tr1, tr2)), axis=1)
__out_atr = wwm_mean_1d(__out_tr, __p_n)
__out_tr, __out_atr
```

Additionally, just like in a regular Python code, we can place `print` statements to explore the state at each execution step:

```
>>> expr = """
... ATR:
... tr0 = abs(@in_high - @in_low)
... print('tr0: ', tr0.shape)
... tr1 = abs(@in_high - fshift(@in_close))
... print('tr1: ', tr1.shape)
... tr2 = abs(@in_low - fshift(@in_close))
... print('tr2: ', tr2.shape)
... @out_tr = nanmax(column_stack((tr0, tr1, tr2)), axis=1)
... print('tr: ', @out_tr.shape)
... @out_atr = wwm_mean_1d(@out_tr, @p_n)
... print('atr: ', @out_atr.shape)
... """
```

```
... @out_tr, @out_atr
...
>>> ATR = vbt.IF.from_expr(expr, n=14)
>>> atr = ATR.run(ohl['high'], ohl['low'], ohl['close'])
tr0: (153, 1)
tr1: (153, 1)
tr2: (153, 1)
tr: (153,)
atr: (153,)
```

WorldQuant's Alphas

IndicatorFactory.from_wqa101 uses the expression parser to parse and execute **101 Formulaic Alphas**. Each alpha expression is defined in **wqa101_expr_config**, while most functions and resolved functions used in alpha expressions are defined in **expr_func_config** and **expr_res_func_config** respectively.

To get an indicator:

```
>>> WQA53 = vbt.IF.from_wqa101(53)
>>> wqa53 = WQA53.run(ohl['open'], ohl['high'], ohl['low'], ohl['close'])
>>> wqa53.out
2019-12-31 00:00:00+00:00      NaN
2020-01-01 00:00:00+00:00      NaN
2020-01-02 00:00:00+00:00      NaN
2020-01-03 00:00:00+00:00      NaN
2020-01-04 00:00:00+00:00      NaN
...
2020-05-27 00:00:00+00:00    0.193719
2020-05-28 00:00:00+00:00   -0.858778
2020-05-29 00:00:00+00:00    0.452096
2020-05-30 00:00:00+00:00    0.376475
2020-05-31 00:00:00+00:00   -0.539368
Freq: D, Length: 153, dtype: float64
```

Or, using a shortcut:

```
>>> vbt.wqa101(53)
vectorbtpro.indicators.factory.wqa101.WQA53
```

Replicating an alpha indicator is quite easy: look up its expression in the config and pass to **IndicatorFactory.from_expr**:

```
>>> WQA53 = vbt.IF.from_expr("-delta(((close - low) - (high - close)) / (close - low), 9)")
>>> wqa53 = WQA53.run(ohl['open'], ohl['high'], ohl['low'], ohl['close'])
>>> wqa53.out
2019-12-31 00:00:00+00:00      NaN
2020-01-01 00:00:00+00:00      NaN
2020-01-02 00:00:00+00:00      NaN
2020-01-03 00:00:00+00:00      NaN
2020-01-04 00:00:00+00:00      NaN
...
2020-05-27 00:00:00+00:00    0.193719
2020-05-28 00:00:00+00:00   -0.858778
2020-05-29 00:00:00+00:00    0.452096
2020-05-30 00:00:00+00:00    0.376475
2020-05-31 00:00:00+00:00   -0.539368
Freq: D, Length: 153, dtype: float64
```

 Python code